



AARHUS
UNIVERSITY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

SpisNemt

A mobile-first lookup recipe experience backed by computer vision and neural networks

2026F68

Name	Studynumber
SW — Frederick Lykkegaard Nielsen	202207768
SW — Rasmus Thykjær Pedersen	202107920

Supervisor: Carsten Eie Frigaard

Pages: 65

Character count: 71.430

Hand in date: May 29th 2026



Electrical and Computer Engineering
Aarhus Universitet

Abstract

This project investigates how to design and implement a mobile application, SpisNemt, that uses client-side machine learning to classify kitchen ingredients and recommend personalized recipes to help mitigate food waste.

The system architecture uses React Native via the Expo Framework along with TensorFlow.js, enabling the execution of neural networks directly on the user's hardware to ensure low latency. Additionally, the application uses a Convolutional Neural Network utilizing an InceptionV3 architecture for ingredient image classification and a Multi-Layer Perceptron (MLP) for recipe recommendation based on user preferences.

The prototype showcases a mobile application that uses machine learning to encourage sustainable habits. Through acceptance testing, it shows high precision for the preference matching of the MLP, along with an accurate computer vision module using the InceptionV3 architecture. Ultimately, the project provides a solid foundation for a tool aimed at encouraging sustainable consumer habits.

Resume

Dette projekt undersøger, hvordan man kan designe og implementere en mobil applikation, SpisNemt, som benytter klientside maskinlæring til at klassificere køkken ingredienser og anbefaling af personlige opskrifter for at hjælpe med at mindske madspild.

Systemets arkitektur benytter React Native gennem Expo sammen med TensorFlow.js, som gør det muligt at eksekvere neurale netværk direkte på brugerens hardware for at sikre lav forsinkelse. Derudover bruge applikationen et Convolutional neuralt netværk ved hjælp af InceptionV3 arkitektur til ingrediens genkendelse og en Multi-Layer Perceptron (MLP) til at anbefale opskrifter baseret på brugerens præferencer.

Projektet demonstrerer en mobil applikation, som benytter maskinlæring til at opfordre bæredygtige vaner. Gennem accepttest påvises der høj præcision for MLP'ens matchning af brugerens præferencer samt et præcist computervision-modul, som bruger InceptionV3-arkitekturen. Projektet danner et solidt fundament for et værktøj, som har til formål at understøtte bæredygtige forbrugervaner.

Contents

1	Preface	4
2	Reading guide	5
3	Wordlist	6
4	Project responsibilities	7
5	Introduction	9
5.1	Problem statement	9
6	Requirements	10
6.1	Functional requirements	10
6.1.1	User stories and moscow	10
6.2	Non-functional	10
6.2.1	Usability	11
6.2.2	Reliability	11
6.2.3	Supportability	11
7	Method and Process	12
7.1	Agile methodology and Scrum	12
7.2	Architectural design and prototyping	13
7.3	Planning and continuous refinement	14
7.4	Risk management	16
7.4.1	Methodology	16
7.4.2	Mitigation techniques	16
8	Analysis	18
8.1	Frontend	18
8.2	Authentication	19
8.3	Database	19
8.4	Machine Learning	20
8.4.1	Ingredient classifier	20
8.4.2	Preference matcher	21
8.4.3	Dataset	21
8.4.4	Version control	22
8.5	Tech stack summary	22
9	Architecture	24
9.1	C4 model	24
10	Design	28
10.1	C3 - Component	28
11	Implementation	37
11.1	Authentication	37
11.2	ErrorBoundary	40
11.3	Machine learning	40

12 Test	53
12.1 Acceptance test	53
13 Results	54
13.1 Acceptance test	54
13.2 Ingredient Classifier Performance	56
13.3 Recipe Recommender Performance	56
14 Discussion	58
14.1 Acceptance test results	58
14.2 Model architecture	59
14.3 The gap between train and test	59
14.4 Reflection on offline capabilities	60
14.5 Omitted requirements	60
14.6 ErrorBoundary	61
15 Conclusion	62
16 Future Work	63

1 Preface

This bachelor thesis was written by Frederick Lykkegaard Nielsen and Rasmus Thykjær Pedersen during the final semester of the Software Technology program at Aarhus University, ECE institute. The project was conducted between the 2nd of February 2026 and the 29th of May 2026.

2 Reading guide

This thesis contain numerous references to figures, tables, and appendices. All of these references are hyperlinks which navigates to the referenced figure, table or appendix. In order for the appendix references to work, the report as well as all appendices must be placed in the same folder and read using a PDF reader. This opens the corresponding appendix when a reference is clicked.

The project has seven appendices:

- A [Appendix Requirements](#)
- B [Appendix Method & Process](#)
- C [Appendix Implementation](#)
- D [Appendix Results](#)
- E [Appendix Discussion](#)
- F [Appendix Future work](#)
- G [Appendix Meetings](#)

3 Wordlist

Acronym	Meaning
API	Application Programming Interface
SDK	Software Development Kit
SQL	Structured Query Language
REST	Representational State Transfer
UI	User Interface
UX	User Experience
YOLO	You Only Look Once
CNN	Convolutional Neural Network
MLP	Multi-Layer Perceptron
EAS	Expo Application Services
FURPS	Functionality, usability, reliability, performance, supportability
DSL	Domain-Specific Language
ReLU	Rectified Linear Unit

Table 1: Thesis Wordlist Definitions

4 Project responsibilities

To present an overview of what each persons responsibility area are, figure 2, provides a table of different areas of the project. An elaboration of each letter in RACI is found in figure 1.

Responsible Does the work to complete the task	Accountable Delegates the work and is the last one to review the task or deliverable before it's deemed complete
Consulted Provides input based on how it will impact their project work or their domain of expertise on the deliverable	Informed Needs to be kept in the loop on project progress, rather than roped into details of every deliverable

Figure 1: Explanation of what the different letter of RACI means

Project Task	Frederick	Rasmus
Design wireframe	R	R
Requirements	R	R
Process management	R	R
Expo App	R	R
Database	I	R
InceptionV3 (ML)	R	I
MLP (ML)	R	C
Accepttest	R	R
Git	R	C
Overleaf	C	R

Figure 2: Table of parts of the project and each members involvement

5 Introduction

In a busy daily life, household tasks like grocery shopping and meal planning are frequently overlooked. Current literature suggests that a primary cause of excessive household food waste stems from consumer behaviors during the period between purchasing and consuming food [1]. Addressing this issue aligns with the United Nations Sustainable Development Goal 12.3, which targets global food loss reduction [2]. Cultivating sustainable habits is critical, as food loss and waste currently account for 8–10% of global greenhouse gas emissions [1].

This project shows how a mobile application for recipe look-up is engineered and developed. It seeks to provide a solution to missing meal planning, which leads to rarely knowing what to cook and eventually wasting food. By letting the user, with their smartphone in hand, scan ingredients and search through a database of recipes, it helps answer the recurring question; "What's for dinner?".

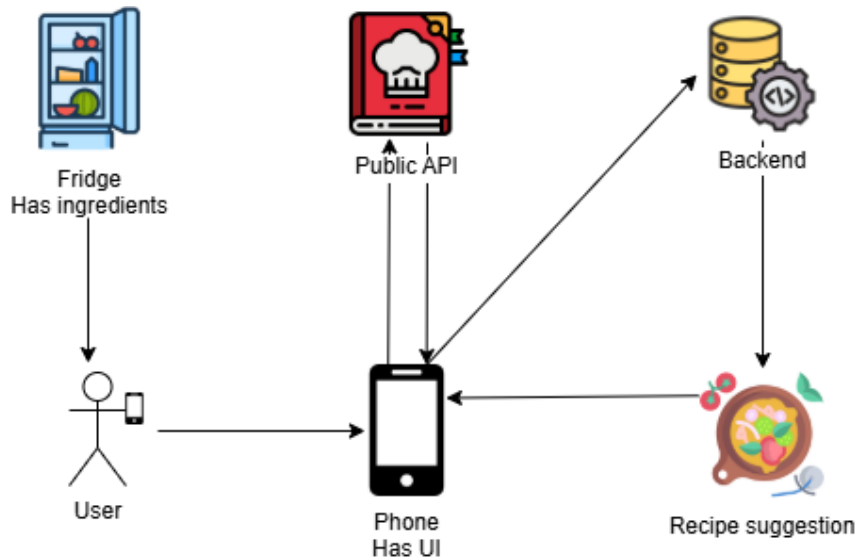


Figure 3: Rich image showing the boundaries of the *SpisNemt* system, mapping the interaction between the user's kitchen ingredients, native hardware views, client-side inference, and remote data APIs.

Figure 3 illustrates the system as a whole. The user takes a picture of their fridge with their smartphone in the SpisNemt app. Using the picture, the ML-model will determine which ingredients the user has available. This data is then used when fetching recipes from the public API, so only recipes the user has the ingredient for, are fetched. The user sets preferences such as preferred cuisine and category, and this is stored in the database and used for filtering returned recipes through the pre-trained ML-model.

5.1 Problem statement

How can a mobile application be designed and implemented using client-side machine learning to classify available ingredients from their kitchen and provide personalized recipe recommendations, thereby offering a tool to mitigate household food waste?

6 Requirements

Stated in this section are the requirements of the system. These said requirements are split into two categories; Functional and non-functional. the functional requirements specify what the system does and how it interacts with users, while the non-functional requirements specify how the system performs while focusing on the FURPS specifications, those being functionality, usability, reliability, performance and supportability.

6.1 Functional requirements

The functional requirements of the system are divided into three epics, and the MoSCoW method is used to grade the importance of each requirement within a user story.

6.1.1 User stories and moscow

Epic 1: User login and registration

- As a user, I **must** be able to register an account and login to said account.
- As a user, I **must** be able to log out of my account.
- As a user, I **must** be able to register preferences such as cuisine and category after user registration.

Epic 2: Ingredient registration

- As a user, I **should** be able to register ingredients as search terms using my device's camera.
- As a user, I **must** be able to register ingredients as text in an input field

Epic 3: Recipes and filtering

- As a user, I **must** be able save recipes.
- As a user, the results of my search **must** be ordered based on match in a descending order from perfect to poor match.
- As a user, I **must** be presented with 4 results at a time, allowing me to refresh the results

6.2 Non-functional

The non-functional requirements of the system are categorized using the FURPS framework, which is a popular choice for organizing non-functional requirements into five categories; Functionality, usability, reliability, performance and supportability. Since functionality is covered in section 6.1, it is left out of the FURPS framework in regards to the non-functional requirements. Unlike the functional requirements, the non-functional requirements specified below are all of equal importance and will not be graded with respect to MoSCoW.

Some original non-functional requirements, with regards to FURPS, have been omitted but can be found in [Appendix Requirements](#). The requirements, based off Lighthouse metrics [3][4][5], are omitted because of lack of relevance to the defined tech stack. This

is discussed in section [14.5](#). As the omitted requirements measures usability and performance, SpisNemt contains no performance requirements.

6.2.1 Usability

- The UI must practice error prevention to minimize the relevance of error handling.
- The system must provide clear error messages.

6.2.2 Reliability

- The user must be able to scan ingredients when offline.
- The user must be able to view their saved recipes when offline.
- The system must be fault tolerant on a UI level, having the ability to continue when one or more components fail.
- The computer vision model must have a validation accuracy of at least 70%.
- Upon classification, the computer vision model must have a confidence of at least 70%.¹

6.2.3 Supportability

- The system must be installable on iOS and AndroidOS.
- The system should be usable as a web application in the user's browser.
- The machine learning model must run on the users device and not through a server.

¹Confidence refers to the highest normalized probability within the model's predicted classes. The confidence in the frontend is observed in Test number 5 shown in table [11](#), and figure [29](#) shows a high training accuracy after training the InceptionV3 image classification model in the Jupyter Environment. More details on the generalization gap between frontend and the Jupyter environment are given in [14.3](#).

7 Method and Process

This section details the methodologies and technical frameworks utilized throughout the development of SpisNemt. To ensure an efficient and iterative workflow, the project follows a tailored Scrum methodology along with the use of the ASE model. The process integrates architectural modeling using the C4 framework and UI/UX prototyping to go from requirements to the final implementation. The following describes how these methods are applied to manage tasks, refine the system architecture, and navigate the phases of the project.

7.1 Agile methodology and Scrum

Scrum was used as a development method for SpisNemt, with weekly sprints planned to support progress. Every Tuesday, members attended a meeting where tasks were assigned to cover one week of work. These tasks were designed to cover one week of work, which was evaluated in collaboration between the members. To keep track of tasks, GitHub Issues was used for SpisNemt, where each member maintained overview of their task through a Kanban board. Tasks were split between report writing and software development, created in collaboration on the basis of discussions between the members. If a task was not completed within the weekly sprint, it was moved to the following sprint. If the opposite happened and the tasks were finished before the end of the week, new tasks from the Kanban board's *todo* column, as seen in figure 4, were pulled into the current sprint. An elaboration of GitHub Issues can be found in [Appendix Method & Process](#).

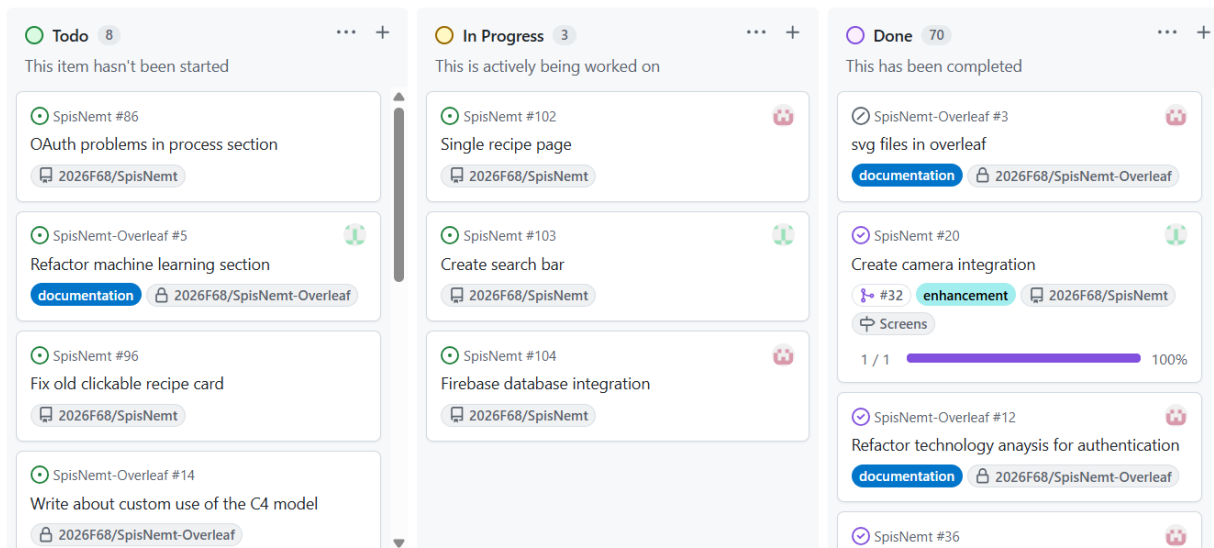


Figure 4: Project Kanban board from the GitHub *Issues* project view, showing issue tracking cards distributed across three statuses: *Todo*, *In Progress*, and *Done*.

According to official Scrum documentation, each sprint concludes with a Sprint Retrospective, an event dedicated to inspecting the team's process and identifying improvements [6]. While retrospective meetings were not fully utilized, this omission opened up for reflection on the development process. In instances where technical 'blockers' arose, such as the problems of the authentication implementation, discussed in [Appendix Discussion](#), a structured retrospective would have facilitated the identification of root causes. By

addressing process performance and necessary adjustments, optimization of the workflow could have been achieved.

The version of scrum utilized for SpisNemt deviated from the official scrum documentation [7], as all group members acted as product owners and developers. This decision was based on the value of collaborative discussion regarding backlog management. If the project had a single product owner, backlog management would be done by a single person who would be accountable for the value delivered. The scrum master role was not used extensively, but was used for ensuring planning and refinement took place which were done in collaboration between team members. A comparison of Scrum used for SpisNemt and official Scrum can be seen in table 2.

Feature	SpisNemt Implementation	Scrum.org (Official)
Product Owner	Shared	Exactly one person
Scrum Master	Shared	Exactly one person
Backlog Mgmt.	Collaborative discussion	PO is accountable
Sprint Length	1 Week	1 Month or less
Daily Meeting	Not utilized	Daily (15 minutes)
Task Movement	Pushed to next sprint if unfinished	Returned to Product Backlog
Retrospective	Not utilized	After each sprint

Table 2: Comparative analysis putting the *SpisNemt* agile implementation up against official *Scrum.org* process standards.

The ASE model is used to define the project along with the implementation of the app. Firstly, the inception phase is quite short as the basic project idea initially starts in the fall of 2025. During the inception phase, requirements are created in order to limit the project to only the most important aspects, where aspects graded "Must" and "Should" are stated. When all requirements are set the elaboration phase begins, resulting in research of what technologies are best suited for the project, based on the requirements made in section 6.2 and user stories from section 6.1.1. This research focuses on reading documentation in order to figure out which technologies work well together and which do not. The chosen technologies are stated in section 8. Additionally, when research is done, sketches are made for SpisNemt which is elaborated in section 7.2.

During the elaboration phase a C4 model is created in order to get an overview of technical elements of the project and how they communicate and interact. The process of creating the C4 model is described in section 7.2 and the full C4 model as well as its containers and components are described in sections 9 and 10.

The construction phase focused on the implementation of the project, where the modified scrum version, as stated in section 7.1, was utilized as support to plan and refine tasks, to help structure the phase.

7.2 Architectural design and prototyping

To facilitate alignment during the design phase, a modified C4 model was used as a tool for an architectural overview. Before implementing any code, the system was hand-drawn on a whiteboard. These analog sessions allowed for multiple iterations where different ways to interpret the system could come to light. Once an adequate analog version was

reached, a digital version was then created using structurizr, elaborated in [Appendix Implementation](#). Throughout the project, this model was iterated upon as mistakes were discovered or the layout needed to be altered to better align with the project's evolution. The C4 model was fully described in section 9 and section 10.

With a C4 model in place, sketches for the GUI were made in the UI prototyping tool Figma [8] to create a visual representation of what the SpisNemt app roughly looks like. This was used as reference during development. These sketches can be seen in [Appendix Method & Process](#). By using this reference, only a small amount of GUI layout decisions had to be made during development. Before creating sketches for the GUI, inspiration was gathered from dribbble [9], where numerous screenshots of mobile food apps are saved and stored in a moodboard in Miro [10]. The moodboard can be seen in [Appendix Method & Process](#).

7.3 Planning and continuous refinement

Refinement was done continuously throughout the development process. Numerous tasks were created as drafts the second an issue came to light, in order to keep track. These draft issues were fully created in collaboration as described in section 7.1. Refinement was not done on planned dates but rather when it naturally came up and both members were physically present.

While internal task refinement was handled continuously, broader project alignment was supported by weekly supervisor meetings. These meetings provided an opportunity to get reviews on and feedback on technical aspects and drafts for report writing. All meeting invitations and meeting summaries can be found in [Appendix Meetings](#)

In the initial phase of the project, A GANTT model was developed, as seen in figure 5, to give a rough time plan for each duration of the project. This time plan was adhered to until the ending of the development phase, as unforeseen obstacles appeared, causing the development phase to be extended by two weeks. This shift in the time plan was made up by the fact that more writing work for the report was done than expected.

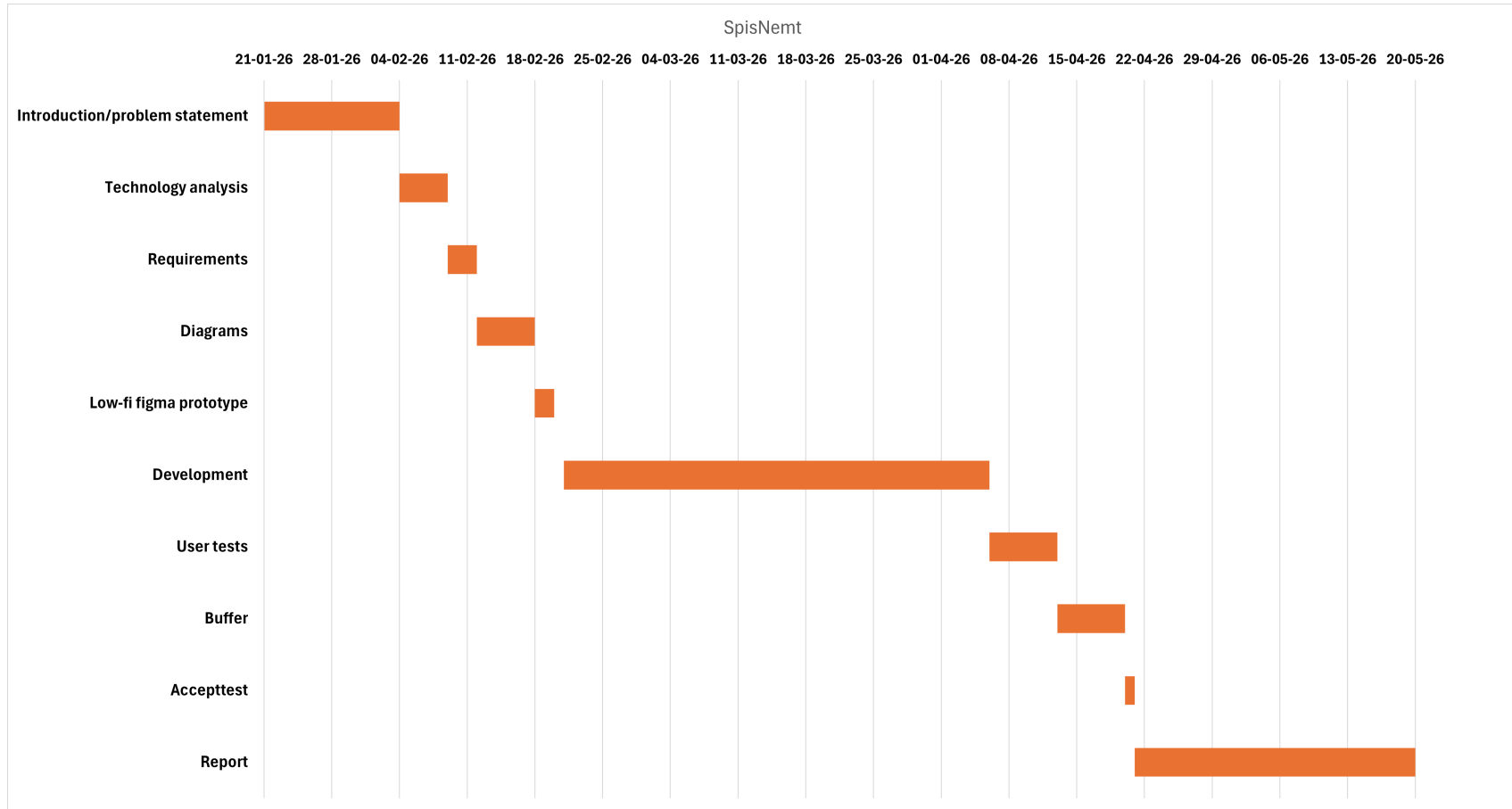


Figure 5: *Gantt chart* timeline mapping the scheduling, milestones, and concurrent work streams for the *SpisNemt* project.

7.4 Risk management

During the development of the *SpisNemt* system, a risk analysis was conducted. Risk management was used to identify potential hurdles and to define mitigation strategies that prevent any hurdles from bringing the development to a halt.

7.4.1 Methodology

The risks were identified in a brainstorming session and categorized into three categories; Technical, Project and External risks. Each risk was evaluated based on impact (I) - that being the severity of the consequences on the success of the project - and the probability - that being the likelihood of the said risk occurring. The metrics were scored on a scale of 1 to 5. The risks were prioritized according to the value coming from the product of the two scores. This allowed for prioritizing which risks required the most attention.

No.	Risk description	Category	I	P	Score	Mitigation strategy
R01	Third party API Complexity	Technical	4	3	12	Prototype of API calls and thorough studying of documentation early
R02	Feature overload	Product	5	3	15	Strictly follow MoSCoW and weekly SCRUM meetings
R03	Backend and Frontend mismatch	Technical	4	2	8	Use shared TypeScript types and a strict feature branching Git strategy
R04	Development vs. Writing time management	Project	5	4	20	Include strict "writing phase" in GANTT and sprint planning
R05	Authentication failure	Technical	5	3	15	Use properly backed, industry standard libraries as opposed to custom code

Table 3: Risk assessment and mitigation priority matrix for the *SpisNemt* project.

7.4.2 Mitigation techniques

Based on the scores in table 3, the following mitigation strategies were implemented to manage the risks of the highest rating in the table.

Development vs. Writing time, score of 20:

The project timeline was managed using a scheduling approach. While a GANTT model provided the initial roadmap, the development phase extended by two weeks due to unforeseen technical complexities. To compensate for this, the team reallocated time from the development buffer to the writing phase, ensuring that the documentation depth was not compromised. This flexibility allowed for the project requirements to be met while maintaining a coherent thesis narrative.

Feature Overload, score of 15:

As noted in section 7.1, the project is split into 1-week sprints. To prevent feature overload, no new features can be added to the sprint once it has started. If a feature is

graded "Could" or "Won't" during the feature refinement, it is moved to the bottom of the GitHub Issues backlog.

Third party API complexity, score of 12:

To mitigate risks stemming from the tech stack, the elaboration phase was utilized to create "Proof of Concept" implementations. This ensures that the architecture described in section 9 is viable, before full-on development phase begins.

8 Analysis

This section contains a systematic evaluation of the technologies that establish the grounding for the technology stack chosen for the SpisNemt system. The technology selection process was driven by the functional and non-functional requirements previously established in sections 6.1 and 6.2.

To ensure high performance, the technological evaluation was divided into five domains: Frontend, Authentication, Database, Machine Learning, and Version Control. By comparing alternatives within each of these domains, this analysis bridges the gap between the initial system requirements and the concrete architectural design detailed in section 9.

8.1 Frontend

While job demand often dictates technology adoption, ecosystem maturity, performance, and developer experience were prioritized for this project. With this in mind, the primary technologies evaluated were either ReactJS, React Native, or a framework based on them.

Given the system requires native hardware access for optimal ingredient scanning, and given the requirements, as stated in section 6.2, specifically regarding usability and reliability, the frameworks Next.js and React Native were the primary candidates.

Next.js enables the development of high-quality web applications using React components through features such as React Server Components, performance optimization through client- and server-side rendering, and advanced middleware capabilities for backend integration [11]. However, the full-stack Expo framework for React Native was selected because its underlying development ecosystem reduces configuration while maintaining support across iOS and AndroidOS, as the same codebase works for both operating systems [12]. Furthermore, Expo is a versatile collection of tools like a CLI for creating and managing projects, a file-system based routing system and an in-house made sandbox app called Expo Go that enables rapid, continuous iteration without native compilation overhead. With the use of native components the development will only target "one" device, thus eliminating the need to access the camera in two different ways for iOS and AndroidOS. A comparison of Next.js and Expo can be seen in table 4.

FURPS Criteria	Next.js by Vercel	Expo for React Native
Usability	High (Mainly web)	High (Native hardware access)
Reliability	Browser-limited	OS integration and native gestures
Performance	High with Server side rendering	High with native build toolchains
Native features	Limited (Web API and libraries)	Primary (Native camera and file system etc.)

Table 4: Architectural technology comparison based on FURPS quality attributes.

To adhere to the requirements of a fault tolerant system at a UI level in section 6.2.2, React's built-in concept *ErrorBoundary* can be used [13]. This makes it possible to track errors more precisely, instead of relying on widespread try-catch statements, by wrapping error prone components in an *ErrorBoundary* element. This catches any error within the component or its children. It is also possible to wrap the entire application in an

ErrorBoundary to notify the user that an error has occurred. Using *ErrorBoundary* enables the ability to have fallback UIs if the original UI fails ensuring the user is not left with a white screen when the app crashes.

8.2 Authentication

The requirements for the project state, in Epic 1 in section 6.1.1, that a user must be able to register an account and login to said account. Different Google sign in methods as well as a custom login system are explored in the following sections.

Expo presents two options for implementing Google login: *Expo AuthSession* [14] and the library *@react-native-google-signin/google-signin* [15]. *AuthSession* is a general purpose authenticator which can be used for Google, Facebook, GitHub and more. The *AuthSession* library is part of the Expo SDK and is therefore part of the Expo Go app. Using the Expo Go app significantly speeds up the time it takes to view changes made during development. However, the official documentation for Expo *AuthSession* [14] recommends using the library supplied by a given identity provider, which in this case is *@react-native-google-signin/google-signin*. In the *Expo AuthSession* documentation the methods for Google- and Facebook login are deprecated and refer to *@react-native-google-signin/google-signin*.

As the recommended approach for OAuth with Expo is *@react-native-google-signin/google-signin*, this was chosen as the login library.

8.3 Database

To store saved recipes as well as user preferences regarding cuisine and category of food, a database was needed. Having already established the use of React Native through Expo in section 8.1, the requirements for the database prioritized offline supportability and compatibility with Expo Go. The Expo documentation provides a section with guides for different kinds of integrations. This includes a guide for using Firebase JS SDK [16], React Native Firebase [16], as well as using Supabase [17]. To ease the process of implementing a database with Expo, the choice was between these three options, since Expo provides guides for all of them.

Firebase JS SDK provides Cloud Firestore which is a database that uses a noSQL approach where data is organized as *Collections* with *Documents* inside of them, typically identified by unique user IDs. Inside of each *Document* is the option to have fields which store data. Firebase JS SDK partially supports offline persistence, so changes made for the user can be stored locally on the device. When a connection is reestablished, all changed data can be committed to Cloud Firestore [18]. This does not happen automatically but only if offline persistence is activated and implemented.

React Native Firebase provides Cloud Firestore as well as the Firebase JS SDK does. This approach offers better offline supportability as offline persistence is enabled by default by creating a SQLite database on the mobile device [19]. All changes made when offline are stored in this database and synchronized with the Firestore database when an internet connection is reestablished. However, React Native Firebase does not offer support for Expo Go and requires the use of development builds, costing significant time during development.

Supabase is built on PostgreSQL and uses SQL as its data language. Supabase exposes PostgreSQL tables in the database. Because of SQL it is easier to filter data in the database contrary to the collection and document structure in Firestore. Supabase however, does not offer offline support, since every response from the database to Expo is stored in memory(RAM), consequently being removed with a reload of the app. It is however possible to persist the data manually with a local database.

Criteria	Firestore	Supabase	React Native Firebase
Query language	noSQL	SQL	noSQL
Offline support	Partially, but not by default	Partially, but harder to implement	Yes, by default
Expo setup guide	Yes	Yes	Yes

Table 5: Comparison of Firestore from Firestore JS SDK, Supabase, and React Native Firebase.

As seen in table 5 the three options are all able to use offline persistence while it is easier with React Native Firebase. However since React Native Firebase does not support the use of Expo Go, Firestore through Firestore JS SDK was chosen as the database. A reason was also that the use of development builds instead of Expo Go would significantly slow down the development speed.

8.4 Machine Learning

8.4.1 Ingredient classifier

Given computer vision and random forest classifiers being essential functions in the system, choosing the right model was crucial. For classifying recipes based on images provided by the user, there were a few ideal candidates. Ideally, the app would allow the user to point their native camera at a densely packed fridge, and the model would classifying the ingredients in the given frame all at once.

For image classification, an ideal fit for detecting multiple objects at once is the YOLO model. The YOLO model is a real-time object detection model, which is high-performing in terms of speed and accuracy. YOLO’s architecture consists of a backbone, neck, and head. The backbone functions as the primary feature extractor and generates feature maps, these maps are aggregated by the neck across different spatial scales to ensure objects of varying sizes can be detected. Lastly, the head serves as a predictor, processing the aggregated features to generate bounding boxes coordinates. It uses a Convolutional Neural Network, creating feature maps, then bounding boxes, and at last labels these bounding boxes. Images are only passed through the model once, making the model faster than other models for these type of tasks. The caveat of YOLO is that the dataset used for training requires bounding boxes on each image. Finding datasets like this is more difficult than regular datasets.

Another options is the Inception V3 system allows for exporting the model to a web-friendly format, letting it be executed client-side via the TensorFlow.js system. After quantizing the model, thus using less resources on the phone, server-side latency is reduced and high performance when classifying is ensured. By using the TensorFlow.js system on

the native hardware of the user’s device, the classification module of the mobile application becomes responsive to an extent that satisfies the criteria established in section 6.1. A comparison of the two models can be seen in table 6.

Metric	YOLO	Inception V3
Goal	Localization	Precise identification
Requirements	Images and bounding boxes	Labeled images
Context	Real-time detection	Static images
Complexity	High w/ implementation and labeling	Moderate w/ transfer learning

Table 6: Comparison between YOLO and Inception V3 Models

While YOLO is superior in real time localization of multiple objects within a single image, the Inception V3 architecture was chosen for the final classifier module. The decision was based on the time horizon of the project, where the accuracy of the classification offered by Inception V3 was prioritized over the speed of real time detection of multiple objects offered by YOLO. By utilizing Inception V3’s feature extraction capabilities, the system achieves a sufficient precision for recipe searching without the time consuming data-annotation required by the YOLO framework.

8.4.2 Preference matcher

For recommending recipes based on the user’s preferences and allergies, a random forest classifier was considered an ideal fit as it is excellent for handling tabular data like the preferences and allergies. However, a random forest classifier requires a separate Python backend service to be implemented and maintained, increasing system complexity and latency. To reduce the supportability burden and satisfy the supportability requirements of 6.2.3, a better fit was determined to be a Multi-Layer Perceptron, since it can be exported and executed on the user’s device alongside the InceptionV3 image classifier. By keeping the recommendation logic on the client-side, it simplifies the system. A comparison of the Random forest and the Multi-layer perceptron can be found in table 7.

Metric	Random forest	Multi-Layer Perceptron
Model type	Decision trees	Neural Network
Environment	Server-side	Client-side
Supportability	Multiple runtimes	Single runtime
User privacy	Data sent to server	On-device locally

Table 7: Architectural comparison between Random Forest and Multi-Layer Perceptron implementations for the recipe recommendation engine.

8.4.3 Dataset

The dataset is arguably more important than the machine learning model itself. A well-structured dataset consists of balanced labels to circumvent bias in the machine learning model. Another indicator is consistent image sizing and file format, creating an easier image pipeline.

Having focused the computer vision to only classify vegetables to downscale the project size, a dataset containing only vegetables had to be used to train the Inception V3 model to align its new head with vegetable recognition during transfer learning. A rule of thumb is that when training a CNN the dataset must have 1000 representative images for each class [20].

8.4.4 Version control

For collaborative development and maintaining reliability of deployments, a feature branching workflow was implemented with Git. This ensures that the production branch stays protected from experimental code, potentially breaking features unlike trunk-based development. Besides that, the integration of forced pull requests and peer review ensures that code is only merged after a peer review. This entire process maximizes the overall supportability of the system as specified in the supportability requirements section 6.2.3. A comparison of trunk-based development and feature branching can be found in table 8.

Metric	Trunk-based development	Feature branching
Stability	Risk of breaking builds	High due to protected main
Code review	Hard to review large changes	Easy review with PRs
Complexity	Simple	Moderate, requires merge management
Best for	Small, fast-moving scripts	Collaborative development

Table 8: Comparative analysis of Git branching strategies across stability, review, and supportability metrics.

8.5 Tech stack summary

Based on the evaluations in table 8, table 9 summarizes the final technologies chosen for the SpisNemt system and the primary deciding factors behind each choice.

Domain	Chosen Technology	FURPS Justification
Frontend	Expo React Native	Single codebase for iOS and Android and native camera access. Satisfies Usability and Supportability requirements.
Authentication	@react-native-google-signin/google-signin	@react-native-google-signin/google-signin is compatible with Expo which is necessary for the system to be installable on iOS and AndroidOS
Database	Firebase Firestore	Offline persistence with seamless Expo integration. Satisfies Reliability and Usability requirements.
Computer Vision	Inception V3	Client-side execution for privacy and low latency. High accuracy via transfer learning.
Recipe Recommending	Multi-Layer Perceptron	Lightweight with 49 parameters. Runs locally, avoiding external server setup. Satisfies Supportability requirements.
Version Control	Feature Branching via Git	Protects the production environment and enforces peer review via pull requests. Satisfies Supportability requirements.

Table 9: System technology stack composition with respects to FURPS architectural justifications.

8.6 Section summary

In summary, the systematic evaluation of technologies establishes a technology stack that fulfills the non-functional requirements detailed in section 6.2.

9 Architecture

This section details the architecture of the SpisNemt system. To satisfy the Reliability and Supportability requirements outlined in section 6.2.2 and 6.2.3, the system uses a decoupled architecture that separates the client-facing frontend, the local machine learning models, and the external third-party API.

9.1 C4 model

To provide an easily read visualization of the system, the C4 model is utilized as the primary architectural framework. The C4 Model consists of four levels; Context, Container, Component, and Code. Each of these layers represents a level of abstraction, where System Context (C1) provides the broadest overview, and Code (C4) provides the most detailed overview of a specific part of the system. By abstracting the system into the three different levels of the C4 model, the documentation ensures that business logic and technical infrastructure are precisely represented. The C4 model for SpisNemt is a modified version of the C4 model, created by Simon Brown, where C4 model documentation [21] is used as reference.

Only three of the four levels of C4 are utilized, as including the code level significantly increases the model’s size without providing additional understanding. The code level is only recommended for the most complex components as stated in the documentation for the C4 model [22]. Another reason for omitting the code level from the C4 model is that it often creates more confusion than insight. The modified version of the C4 model deviates from the documentation as no labels are used for the transport layer between boxes in each diagram. This is done for readability and because all communication between boxes is explained in section 9.1.2 and 10.1.1. Another difference between an original C4 model and the one used for SpisNemt is that no descriptions of the boxes appear in the model as specified in the documentation’s ”Elements” section [23]. The reason for not including descriptions is that each box is described textually in sections 9 and 10.

9.1.1 C1 - Context

The context level provides the highest level of abstraction, illustrating how SpisNemt interacts with external actors and systems. As shown in figure 6, the user interacts directly with the SpisNemt application to receive their recipe recommendations. The system relies on two external entities, presented with a gray color: TheMealDB, which is an external third-party API for recipe data, and an external database for storing user profiles and users’ respective preferences as well as their saved recipes.

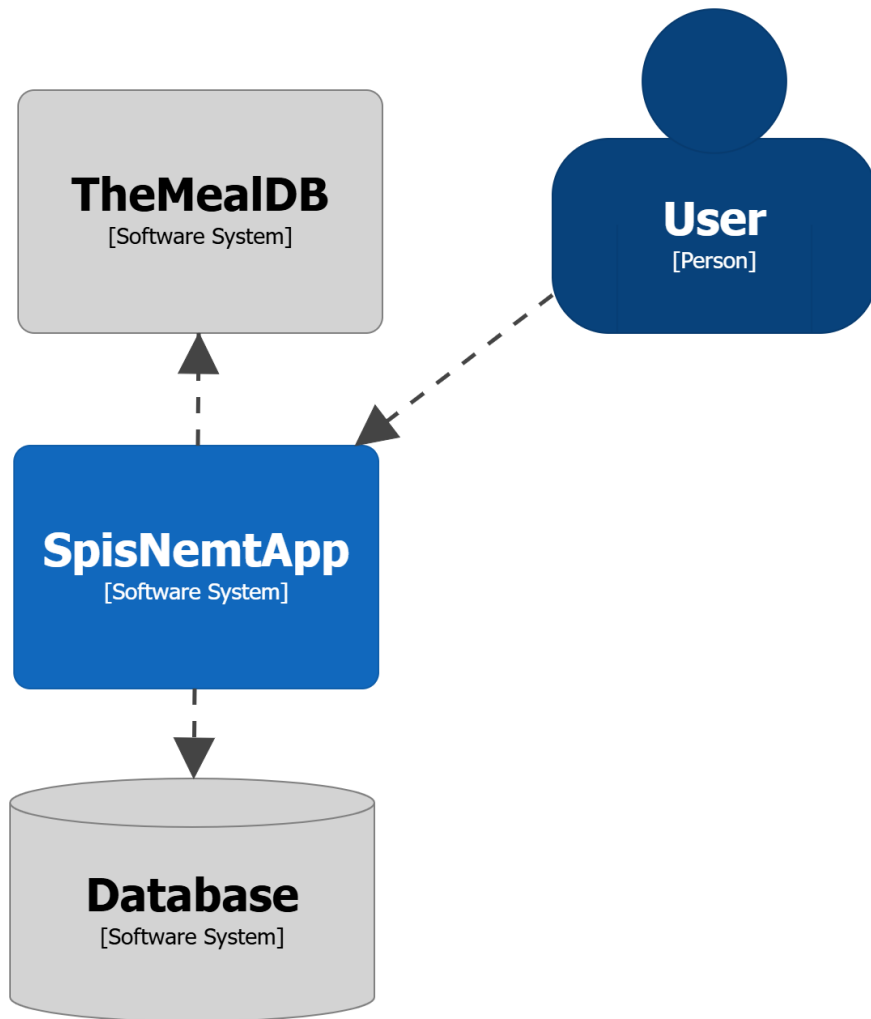


Figure 6: C1 System Context diagram for the SpisNemt ecosystem, illustrating boundaries between the primary client application, the end user, and external infrastructure dependencies.

9.1.2 C2 - Containers

At the container level, the system is decomposed into five containers, as shown in figure 7. The UI container manages all of SpisNemt's screens, which represent the different views of SpisNemt. The Backend container represents a SignInController, a PreferenceService, and the machine learning models, specifically the RecipeRecommender and ComputerVision. MealDBService and UserService are containers responsible for communication with the external software systems, specifically TheMealDB and the Database. FirebaseAuth is used in collaboration with the backend to authenticate users when logging in, updating their preferences, or saving a recipe.

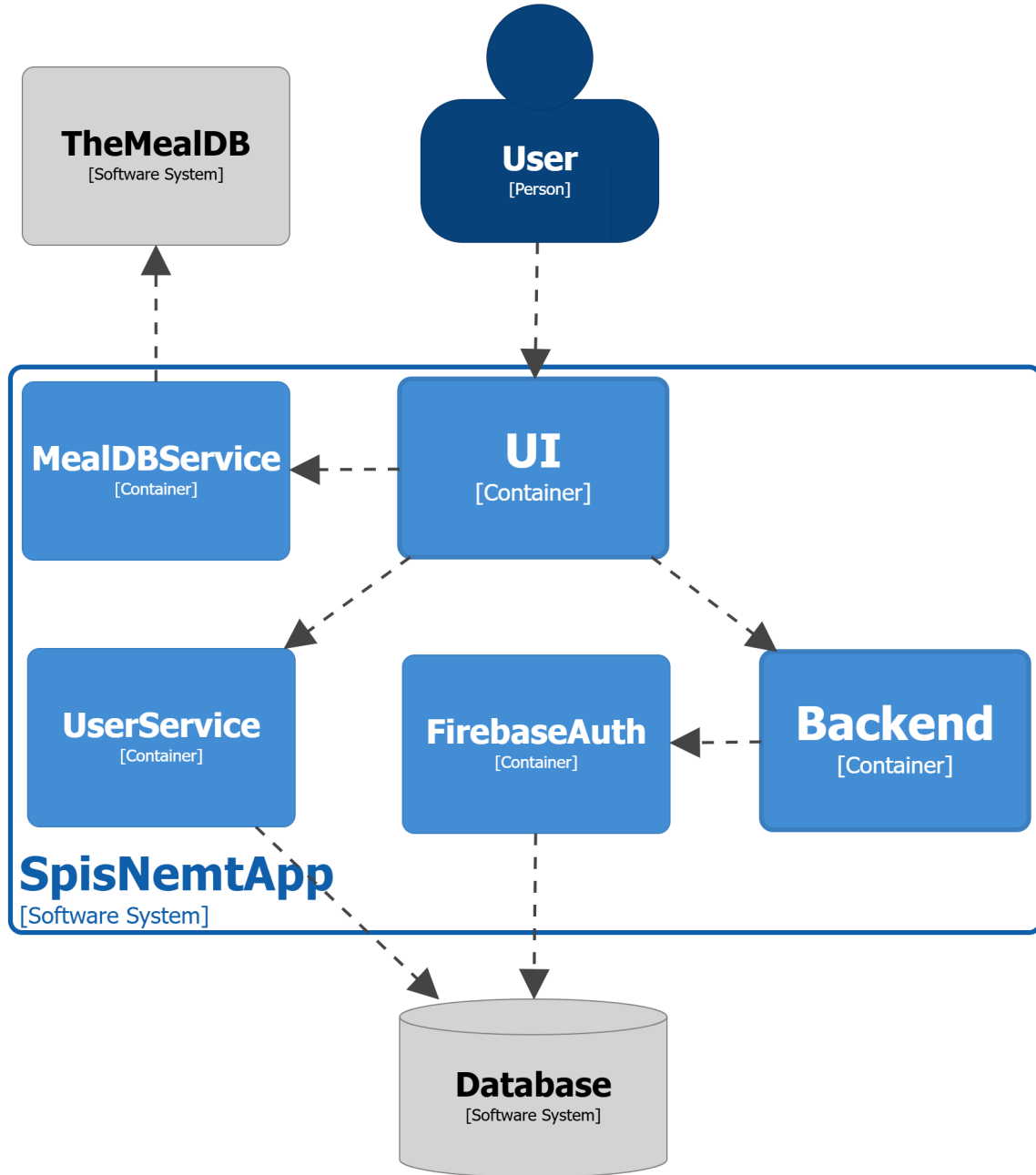


Figure 7: C2 Container diagram for the SpisNemt system, mapping the structural boundaries, separation of concerns, and communication protocols between the presentation layer, internal services, and external platforms.

To communicate between the UI and the Database, Firebase Firestore SDK [24] methods are used. These include methods such as *getDoc()* and *setDoc()*, used to fetch user information and update it, respectively. Communication between the UI and the Database occurs through the **UserService** where different methods for updating data using the *setDoc()* method are defined. RESTful APIs are used between the UI and **TheMealDB**. All communication from the UI to **TheMealDB** occurs through the **MealDBService**, where methods for fetching recipes, cuisines, and categories are defined.

9.2 Section summary

To conclude, by dividing the system into the five containers, the architecture isolates external dependencies from internal application logic. Additionally, by routing data through services and dividing the system into UI and Backend containers, a separation of concerns is established.

10 Design

Following the system overview of the architecture in section 9, this section provides details on the internal design of the software through the remaining C3 level of the C4 model. The objective is to demonstrate the internal logic of the containers from figure 7, and what each component is responsible for as well as how they communicate and interact with each other.

10.1 C3 - Component

The component level of the C4 model demonstrates the internal structure of the containers from the C2 diagram in figure 7, by "exploding" the view of a container from the container diagram. Each block inside a container is called a component. The following sections contain descriptions of the UI and Backend component diagrams. No diagrams of the remaining UserService, MealDBService, and FirebaseAuth exist as the main interest lies with the UI and Backend. UserService and MealDBService are simply a collection of functions which call the database and TheMealDB API respectively and FirebaseAuth provides authentication functionality through a variety of functions from the Firebase JS SDK.

10.1.1 C3 Component view - UI

To show what happens at a UI level figure 8 includes a component for each of the screens in SpisNemt. The main focus centers around four screens: HomeScreen, ExploreScreen, SavedRecipesScreen, and AccountScreen. Using these screens SpisNemt is divided into different areas, following a common navigation bar layout in modern apps.

The HomeScreen is where recommended recipes as well as recipes sorted by category are shown, so the user can quickly choose a recipe which catches their interest. Clicking a recipe displays a bottom sheet² containing all information about a recipe like ingredients and instructions. Recipes can be saved using the bookmark button.

ExploreScreen is where ingredients are scanned using the phone camera through a native camera component. These ingredients are scanned by the ComputerVision component inside the Backend container, where an in-depth explanation of the Backend is found in section 10.1.2. Once scanned, they are added to the search terms underneath the search bar. A second way to add search terms is using the search field, where they are simply written in an input field. The search terms are used by the RecipeRecommender, which can be read about in section 10.1.2

The SavedRecipesScreen displays all the recipes which the user has saved through the bookmark feature when viewing a recipe. These recipes are fetched from the database where the id of the recipe is stored. The SavedRecipesScreen calls TheMealDB through MealDBService with the id to fetch the actual recipe.

²A bottom sheet is a surface anchored to the bottom of the screen, that slides up to reveal additional content or details

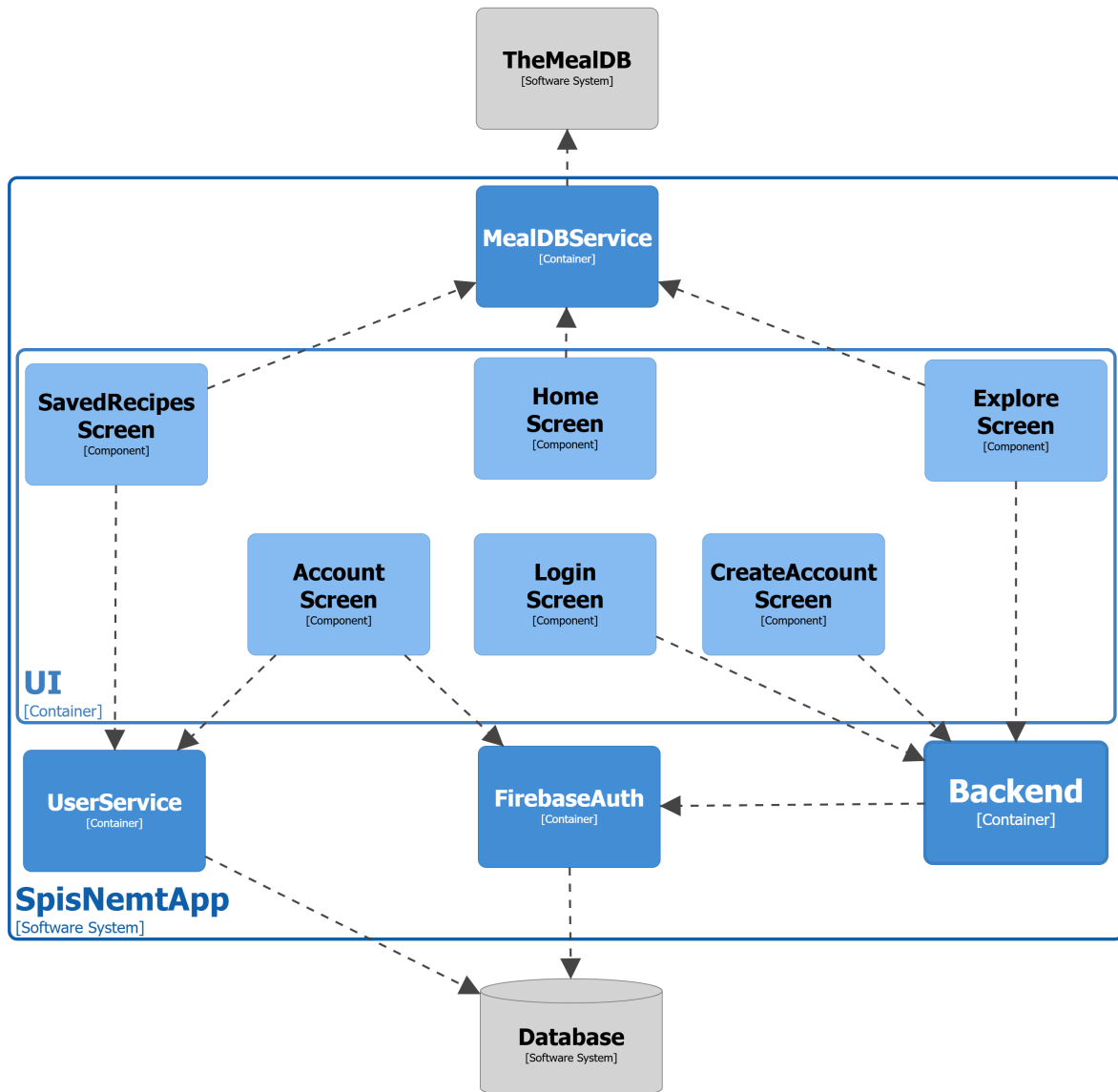


Figure 8: C3 Component diagram detailing the presentation layer architecture of the UI container, illustrating view isolation, cross-container data routing, and interaction pathways.

The AccountScreen displays information about the current user as well as the user's preferences in terms of cuisine and category. The user is able to edit their preferences in the AccountScreen. A sign out button to sign out the user is located in the AccountScreen as well.

LoginScreen is used for account authentication, where a user can choose to login to an existing account or navigate to the CreateAccountScreen in order to register a new account. As seen in figure 8, LoginScreen and CreateAccountScreen both use the Backend container to communicate with FirebaseAuth to register a new user in the database or login to an existing user. In the backend a SignInController is used, which is described in section 10.1.2.

HomeScreen, ExploreScreen, and SavedRecipesScreen all communicate through the MealDB-Service by *fetch()*-methods in order to access recipes, areas, or categories from The-MealDB. Likewise, SavedRecipesScreen and AccountScreen communicates through UserService in order to access account data such as preferences and saved recipes. This communication happens through the Firebase Cloud Firestore SDK, which provides a *DocumentReference* to the logged in user, which is updated when preferences are set or recipes are saved. MealDBService and UserService are elaborated in [Appendix Implementation](#).

10.1.2 C3 Component view - Backend

The Backend container serves as the core processing engine of SpisNemt, encapsulating the system's business logic, data management, and machine learning inference. As shown in figure 9, this container is decomposed into four primary components that collaborate to provide the services consumed by the UI container.

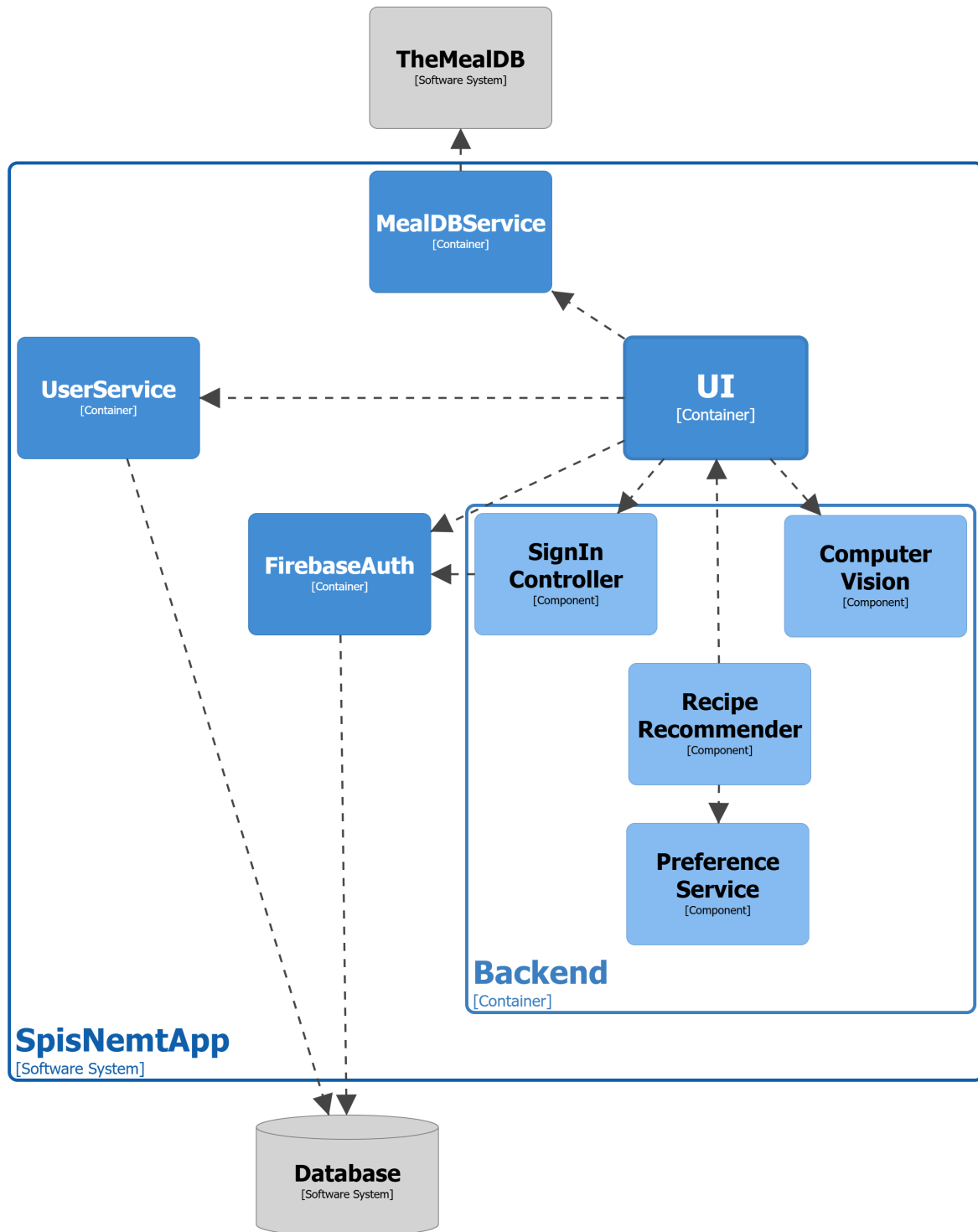


Figure 9: C3 Component diagram showing the internal sub-modules of the Backend container and their directional communication lines with their architecture containers.

SignInController

The SignInController is responsible for authenticating users when logging into their account as well as creating new accounts. When creating a new user, a form consisting of a name, email, and password must be filled out. Email and password are used to log in and creating a new account automatically signs in with the new account.

Authentication is done by Firebase Authentication which is used in collaboration with Cloud Firestore. The SignInController holds a state that changes when a user logs in or out, this state is also capable of checking if a user is already logged in when opening the app. When logged in to an account all communication to and from the database happens in relation to said account's Firestore document.

RecipeRecommender

To satisfy requirements stated in Epic 3 of section [6.1.1](#) regarding matching user preferences, an MLP model is built as a feed-forward neural network. This MLP consists of three layers: An input layer, a hidden layer, and an output layer. In the input layer, there are 4 input neurons to represent match quality features between what the user wants and what a given recipe provides. The hidden layer is a dense layer with 8 neurons, which is visualised in figure [10](#). This layer utilizes the ReLU activation function, allowing the network to learn non-linear relations between the input features. Choosing a small number of neurons ensures the model stays lightweight by totaling 49 trainable parameters. This architecture is intentional, as it enables efficient native performance, helping satisfy the supportability requirements in [6.2.3](#).

The hidden layer is connected to an output layer consisting of a single neuron with a sigmoid activation function. This function outputs the model's calculations into a probability-like score between 0 and 1. In the context of the SpisNemt system, a score closer to 1 shows a high probability that the recipe aligns with the user's saved cuisine and category preferences, while a score of 0 suggests a very poor match.

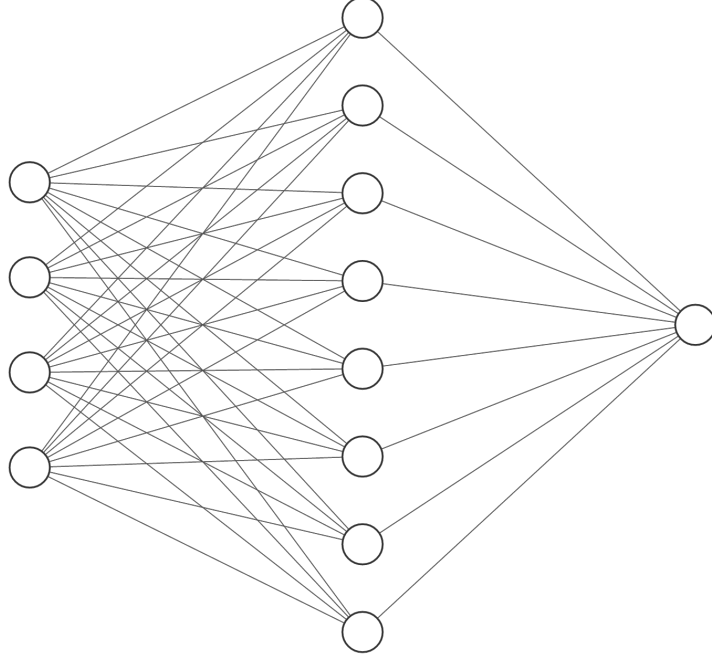


Figure 10: Structural diagram of the feed-forward MLP engine, illustrating the $4 \rightarrow 8 \rightarrow 1$ fully connected neural network layers utilized by the local client inference system.

To define what should be considered a "perfect", a "good" and a "poor" match, a labeling rule is established to score a recipe based on the input features. The labels are generated based on a weighted scoring function S , defined as:

$$S = 60x_{area} + 40x_{cat} \quad (1)$$

where x_{area} and x_{cat} represent the binary match signals for the area and category. A target label of 1 for a positive match is assigned if $S \geq 40$, ensuring that any single feature match results in a positive classification.

To calculate the final probability, that being how likely it is for the user to like a given recipe, the MLP performs mathematical operations across each of the layers. Each of the dense layers transforms the input vector using a weight matrix $\mathbf{W}$ and a bias vector $\mathbf{b}$, followed by an activation function ϕ :

$$\mathbf{z} = \mathbf{W} \cdot \mathbf{x} + \mathbf{b} \quad (2)$$

where $\mathbf{W}$ represents the weight matrix, $\mathbf{x}$ is the input vector from the previous layer, and $\mathbf{b}$ is the bias vector. After this, an activation function ϕ is used on the weighted sum to produce the output y of the layer:

$$y = \phi(\mathbf{z}) \quad (3)$$

In regards to the hidden layer, the ReLU activation function is used to handle the weighted sums. This function helps the network learn which feature combinations are the most important:

$$\phi(z) = \max(0, z) \quad (4)$$

Lastly, the output layer uses the Sigmoid activation function to fit the result into the range of 0 to 1, returning a probability-like score:

$$\phi(z) = \frac{1}{1 + e^{-z}} \quad (5)$$

The result is the final match score used by the system to rank recipes according to the user’s preferences. By using these equations, the model can generalize from the simple binary rules used for training to provide an actual nuanced probability for new recipe recommendations.

PreferenceService

To build the input channel required by the MLP, described in section 10.1.2, a Preference-Service is designed within the backend container. This service acts as an abstraction and data transformation layer that maps raw unstructured user data stored in the database to the uniform vectors required for client-side tensor execution.

The architectural composition of this componenet revolves around three central tasks; lifecycle asset management, binary vectorization mapping, and tensor execution tracking.

To satisfy the system supportability guidelines outlined in section 6.2.3, the service relies on a client-side lifecycle. Rather than doing asynchronous network requests to an external server, the service bundles the underlying neural network parameters natively inside the application bundle using the *@tensorflow/tfjs-react-native runtime provider*.

To maintiain responsiveness of the application, an initialization subsystem was structured as a lazy-loaded wrapper during the development phase. Upon execution start, the service validates and compiles the model.json definitions alongside its binary weight shards. This routine follows a strict initialization constraint function where the initialization time must be less than or equal to a timeout threshold of 10.000 ms. If initialization crosses this threshold, the operational instance is safely discarded, resetting the state to prevent UI freezes.

Once the model instance is verified, the service takes real world information from the app and converts it to data that the machine learning model can understand. It accepts an input tuple consisting of an unranked recipe data object, *MealForScoring*, and a structured tracking object containing the user preference profile arrays, *UserPreferences*. The service executes an inclusion mapping function to establish the binary match elements *x_area* and *x_cat* based on whether the candidate meal’s area or category properties are included within the user’s non-empty preference arrays. If the preference array contains items and the candidate property matches an included string, the signal maps to 1.0. If not, it defaults to 0.0.

Following the derivation of these match values, the service normalizes and builds a static target input data array containing four float elements. The array duplicates the computed binary values to match the precise sequence expected by the model layout:

inputData = [*areaMatch*, *catMatch*, *areaMatch*, *catMatch*]

This data vector is converted into a two-dimensional tensor containing one row and four columns to match the input shape expected by the model. This tensor passes directly into the model’s prediction function. Following the generation of the match score, the memory allocated for both the input and output tensors is explicitly cleared. This memory cleanup prevents performance degradation and ensures a minimal hardware footprint on the mobile device during intensive recipe ranking operations.

Computer Vision

To fulfill the requirements for ingredient registration through the device’s camera as defined in Epic 2 in section 6.1.1, an Ingredient Classifier is designed using a Convolutional Neural Network (CNN) architecture. Additionally, given the reliability requirements, in section 6.2.2, and the complexity of identifying food items, the system uses a Transfer Learning strategy built on the InceptionV3 architecture.

The Inception V3 model is selected for its high-performance feature extraction, which allows the model to classify food items such as fruits and vegetables by capturing micro-details like textures of the skin and macro-features like the overall shape.

The architecture of the model is implemented in modules as follows: For input scaling, the model is designed to accept images with a fixed dimension of 299 x 299 x 3 pixels. The base model is initialized with weights pre-trained on the ImageNet dataset. The base layers are frozen to avoid overwriting the unique feature extraction capabilities of InceptionV3 during training.

Lastly, to improve the model’s ability to generalize across different environments - that being kitchens that might vary from user to user - a sequential augmentation layer is added. This layer incorporates random horizontal and vertical flips, a 20% random rotation and a 20% random zoom.

Regarding the mathematical operations, the classifier identifies ingredients by applying filters to the input image. Universally, this operation is referred to as convolution in deep learning frameworks as cross-correlation. It is similar to mathematical convolution, but without the mirroring of the filter. In the implementation, this filter slides on top of the pixel grid to produce feature maps that identify visual patterns such as textures in the micro-details and shapes in the macro-features.

In order to adapt the Inception model to recognizing food items, a custom classification head is implemented. This head consists of a Global Average Pooling layer to reduce the three-dimensional feature map to a one-dimensional vector. This vector is followed by a dropout layer to prevent overfitting.

The final output is produced by a Dense layer using the Softmax activation function, which normalizes the outputs into a probability score where the sum of all classes adds up to 1. The probability ϕ for a class i is calculated like such:

$$y_i = \phi(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (6)$$

where z_i is the raw output for each of K vegetable categories. As specified in the non-functional requirements of reliability in section 6.2.2, the system only registers an ingredient if the resulting score is above the 70% threshold.

10.2 Section summary

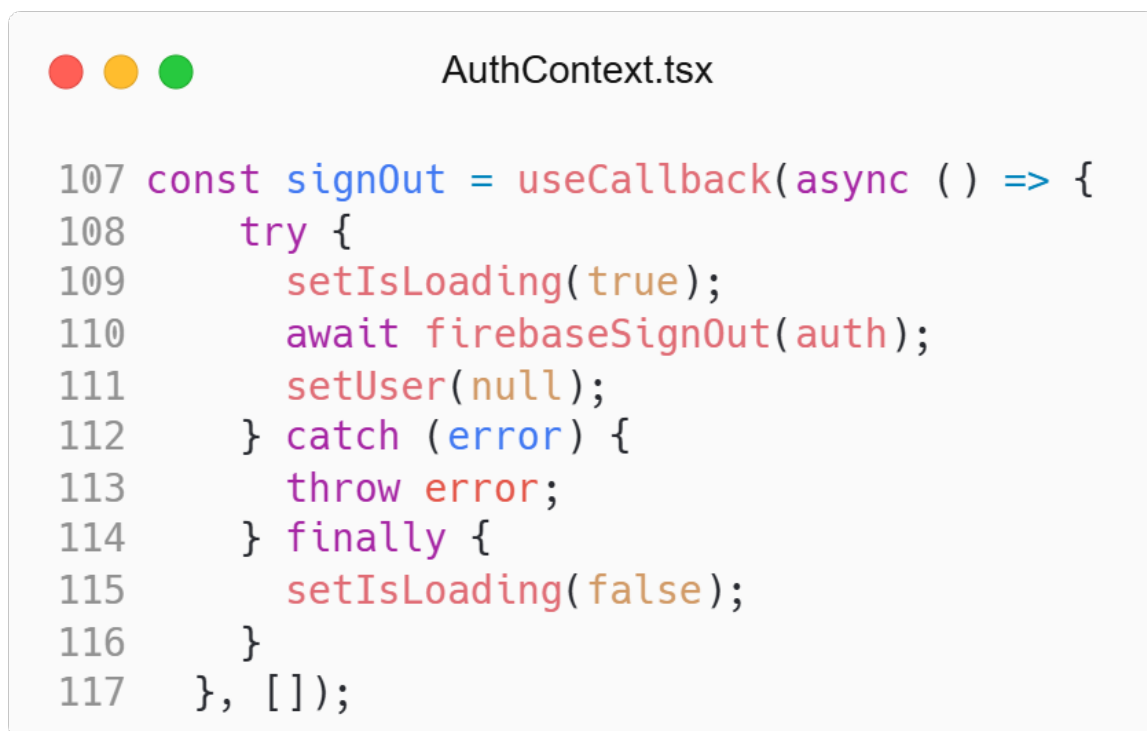
Summarizing, by keeping the UI and Backend containers in specialized components, the design of SpisNemt ensures a clear separation of concerns. The frontend components handle user interaction and navigation, while the backend components encapsulate the mathematical logic required for on-device machine learning inference. Low fidelity sketches are made in Figma.

11 Implementation

This section describes the realization of the SpisNemt system, translating the theoretical architecture and component designs defined in sections 9 and 10 into functional software. The following sections demonstrate translation of the UI layouts, the modeling of the C4 diagrams, the integration of Firebase Authentication, and the implementation of the on-device machine learning pipelines using TensorFlow.js. Implementations of C4 Model as well as UserService and MealDBService from the C4 Model can be found in [Appendix Implementation](#).

11.1 Authentication

After switching from *@react-native-google-signin/google-signin* to *AuthSession* to a custom login system using Firebase Authentication, which is discussed in section [Appendix Discussion](#), methods such as *createAccountWithEmailAndPassword*, *signOut*, and *signInWithEmailAndPassword* were integrated from the *firebase/auth* library. The *SignOut* method implementation can be seen in figure 11. The *SignIn* method uses the same structure as *signOut*.

A code editor window titled 'AuthContext.tsx' with three colored window control buttons (red, yellow, green) in the top left corner. The code is a TypeScript function definition for 'signOut' using 'useCallback' and 'async' syntax. It includes a try-catch-finally block to handle Firebase sign-out operations, setting loading states and user information.

```
107 const signOut = useCallback(async () => {  
108     try {  
109         setIsLoading(true);  
110         await firebaseSignOut(auth);  
111         setUser(null);  
112     } catch (error) {  
113         throw error;  
114     } finally {  
115         setIsLoading(false);  
116     }  
117 }, []);
```

Figure 11: Code snippet of the asynchronous *signOut* routine within *AuthContext.tsx*.

The Authentication flow uses a *onAuthStateChanged* function which subscribes to Firebase. Firebase then checks if a session already exists on the device, which then returns a user object if true and null if not.

When creating a new account through the account creation form, the provided *firebase/auth* method *createUserWithEmailAndPassword* is called which registers the new account Firebase Authentication and a user document is created in Cloud Firestore with the *SetDoc* method, as seen in figure 12, provided by the *firebase/firestore* library.



```
5 export async function createUserProfile(user: UserIdentity) {
6   await setDoc(
7     userProfileDocRef(user.id),
8     {
9       email: user.email,
10      name: user.name,
11      preferences: {
12        area: [],
13        category: [],
14      },
15      savedRecipes: [],
16      updatedAt: serverTimestamp(),
17    },
18    { merge: true },
19  );
20 }
21
```

Figure 12: Code snippet of the asynchronous *createUserProfile* function within *createUserProfile.ts*, establishing the foundational schema matrix for newly provisioned user records in Cloud Firestore.

When signing in to an existing account through the login form, *signInWithEmailAndPassword* is called. If successful, *Firebase Auth* updates the user, which sets the *isAuthenticated* boolean to true since it is derived from whether or not a user exists. As for signing out the function *signOut* is called which clears the session, removing the user and sets the *isAuthenticated* boolean to false. As a result all screens guarded by *isAuthenticated* are no longer accessible, thus redirecting to the login screen that is not guarded by *isAuthenticated*. The *isAuthenticated* variable depends on whether or not a user is signed in as seen in figure 13.



```

119 const value = useMemo(
120   () => ({
121     isAuthenticated: Boolean(user),
122     isInitializing,
123     isLoading,
124     user,
125     signIn,
126     createAccount,
127     signOut,
128   }),
129   [createAccount, isInitializing, isLoading, signIn, signOut, user],
130 );

```

Figure 13: Code snippet within *AuthContext.tsx* showing the memorization of the global authentication state object via the *useMemo* hook to avoid triggering unnecessary re-renders across the screen's visual layout.

In order to create a connection between Cloud Firestore and SpisNemt, a configuration file for firebase was created. Keys and ID's are stored in a *.env*-file, and these keys and ID's variables are used in *firebaseConfig.ts* as seen in figure 14.



```

11 const firebaseConfig = {
12   apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY,
13   authDomain: process.env.EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN,
14   projectId: process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID,
15   storageBucket: process.env.EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET,
16   messagingSenderId: process.env.EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
17   appId: process.env.EXPO_PUBLIC_FIREBASE_APP_ID,
18   measurementId: process.env.EXPO_PUBLIC_FIREBASE_MEASUREMENT_ID,
19 };
20
21 const app = getApps().length > 0 ? getApp() : initializeApp(firebaseConfig);
22
23 let auth: Auth;
24 try {
25   auth = initializeAuth(app, {
26     persistence: getReactNativePersistence(AsyncStorage),
27   });
28 } catch {
29   auth = getAuth(app);
30 }
31
32 export const db = getFirestore(app);
33 export { auth };

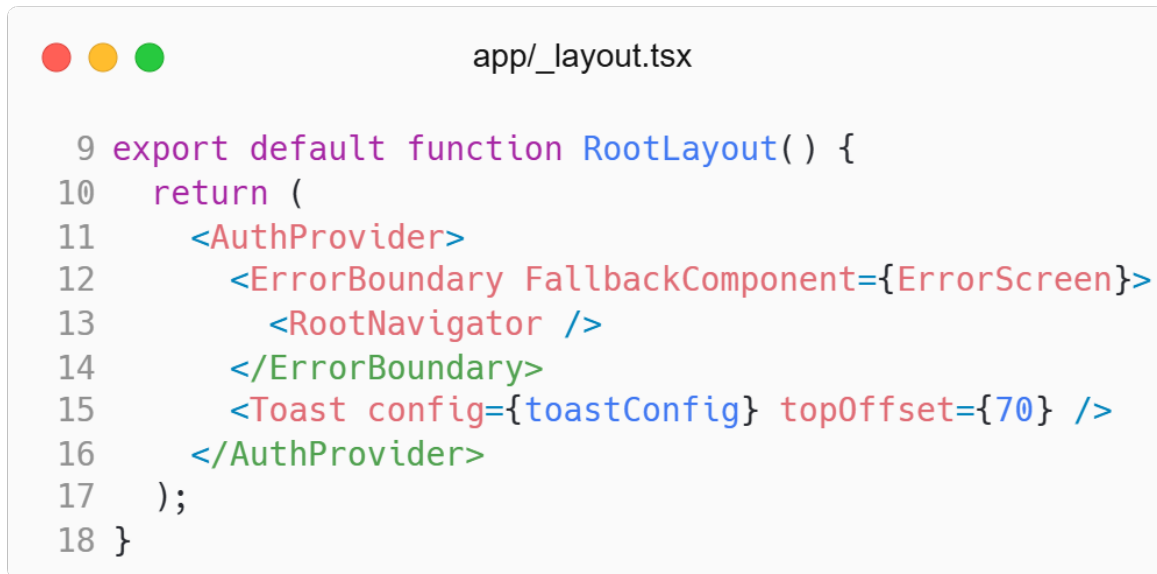
```

Figure 14: Code snippet within *firebaseConfig.ts*, demonstrating structural SDK instantiation and environment variable binding

11.2 ErrorBoundary

As stated in the usability requirements in sections 6.2.2 of the non-functional requirements, the UI must practice error prevention to minimize the relevance of error handling and the system must provide clear error messages.

Through the library *react-error-boundary*, a global UI-level fault-tolerance mechanism is implemented, as seen in figure 15. The *ErrorBoundary* wraps the *RootNavigator*. This placement ensures that unexpected rendering failures within the navigator are contained, thus not immediately terminating the app.

The image shows a code editor window titled 'app/_layout.tsx'. It contains a TypeScript function 'RootLayout' which is the default export. The function returns a JSX element. This element is a nested structure: an outer '<AuthProvider>' wrapper, followed by an '<ErrorBoundary>' component. The '<ErrorBoundary>' component has a 'FallbackComponent' prop set to '{ErrorScreen}'. Inside the '<ErrorBoundary>', there is a '<RootNavigator />' component. After the '<ErrorBoundary>', there is a '<Toast>' component with 'config={toastConfig}' and 'topOffset={70}' props. The entire structure is closed with '</AuthProvider>'. The code is as follows:

```
9 export default function RootLayout() {
10   return (
11     <AuthProvider>
12       <ErrorBoundary FallbackComponent={ErrorScreen}>
13         <RootNavigator />
14       </ErrorBoundary>
15       <Toast config={toastConfig} topOffset={70} />
16     </AuthProvider>
17   );
18 }
```

Figure 15: Implementation of a global UI-level *ErrorBoundary* wrapping the *RootNavigator* in the root layout and calling a *FallbackComponent* if *ErrorBoundary* catches a React rendering error.

When a runtime error occurs, the *ErrorBoundary* switches from rendering the normal UI to rendering a dedicated fallback screen, called *ErrorScreen*, which presents a generic text being "Something went wrong" as well as an error message derived from *error.message* if available. The *ErrorScreen* also provides a "Try again" button which resets the *ErrorBoundary* and attempts to re-render the failed UI. The fallback UI is provided through the *FallbackComponent* prop.

11.3 Machine learning

11.3.1 Preference matching

To implement matching recipes to users based on their cuisine and category preferences, the system uses a Multi-Layer Perceptron to score recipes coming from TheMealDB API, by checking if the cuisine and category of the returned recipe aligns with the users interests. This functionality realizes the Recipe Recommender component defined in the C3 backend architecture of figure 9. The initial setup in the respective Jupyter environment before starting the setup involves setting 3 constants; The EPOCHS of 15, the BATCH_SIZE of 64, the VAL_SPLIT of 0.2, and the N_SAMPLES of 10.000.

Training data generation

The scoring system of the MLP is based on a specific logic used to generate the training data, which is shown in figure 16.

The preference matcher logic stems from the generation of training data. To make sure the MLP ranks recipes correctly, a weighted scoring function S was established during the training phase, as shown in equation 1.

```
MLP_preferencematch.ipynb

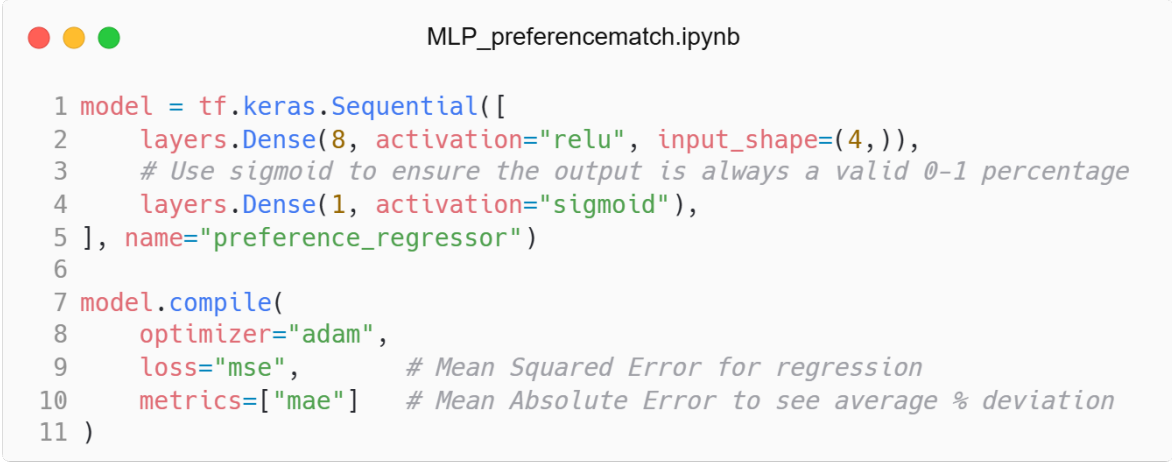
1 import numpy as np
2
3 rng = np.random.default_rng(42)
4
5 def generate_sample():
6     # Simulate a recipe: one area, one category
7     recipe_area_idx = rng.integers(0, N_AREAS)
8     recipe_cat_idx = rng.integers(0, N_CATEGORIES)
9
10    # Simulate user preferences: 0-5 areas, 0-5 categories
11    n_user_areas = rng.integers(0, 6)
12    n_user_cats = rng.integers(0, 6)
13    user_areas = set(rng.choice(N_AREAS, n_user_areas, replace=False).tolist())
14    user_cats = set(rng.choice(N_CATEGORIES, n_user_cats,
15 replace=False).tolist())
16    # Binary match signals
17    area_match = 1.0 if recipe_area_idx in user_areas else 0.0
18    cat_match = 1.0 if recipe_cat_idx in user_cats else 0.0
19
20    x = np.array([area_match, cat_match, area_match, cat_match], dtype=np.float32)
21
22    # Weighted Regression Label
23    # Category = 0.6, Area = 0.4
24    y = (cat_match * 0.6) + (area_match * 0.4)
25
26    return x, y
27
28
29 samples = [generate_sample() for _ in range(N_SAMPLES)]
30 X_train = np.stack([s[0] for s in samples])
31 y_train = np.array([s[1] for s in samples], dtype=np.float32)
32
33 print(f"X_train shape: {X_train.shape}")
34 print(f"Label distribution: 0.0={float((y_train==0).mean()):.2f},
35 0.5={float((y_train==0.5).mean()):.2f}, 1.0={float((y_train==1.0).mean()):.2f}")
36
```

Figure 16: Code snippet within *MLP_preferencematch.ipynb*, mapping the structural vector transformations used to generate data for model training.

The code defined the target label y by assigning equal weight to the matches of the area of a recipe and the category of a recipe. This results in three target scores; 1.0 for "Perfect" matches where both criteria are met, 0.5 for "Good" matches and 0.0 for "Poor" matches.

Model Architecture and training pipeline

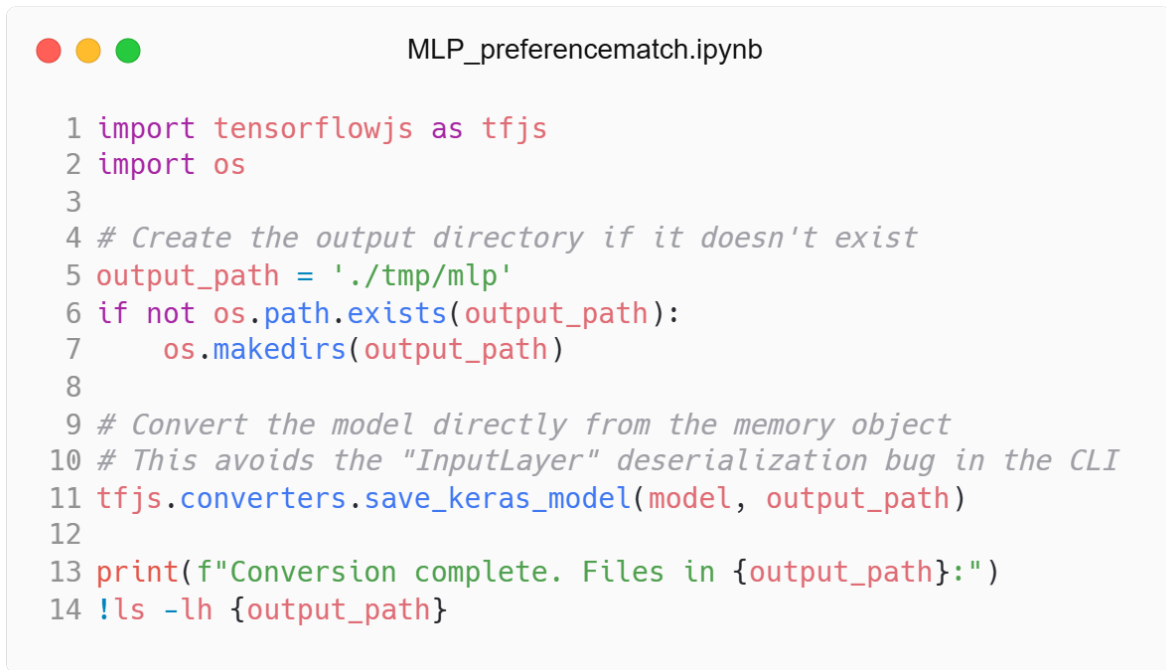
The Multi-Layer Perceptron was implemented and trained in a Jupyter environment using the *Python Keras* library. To satisfy the requirement for client-side execution specified in section 6.2.3, the Sequential API was used to define a minimal architecture; A hidden dense layer with 8 neurons utilizing a ReLU activation function, followed by a single-neuron output layer. The output layer uses a Sigmoid activation function to ensure that the predicted regression score is a number between 0 and 1, representing a match percentage that can later be used in the UI. A code snippet can be seen in figure 17.



```
1 model = tf.keras.Sequential([
2     layers.Dense(8, activation="relu", input_shape=(4,)),
3     # Use sigmoid to ensure the output is always a valid 0-1 percentage
4     layers.Dense(1, activation="sigmoid"),
5 ], name="preference_regressor")
6
7 model.compile(
8     optimizer="adam",
9     loss="mse",      # Mean Squared Error for regression
10    metrics=["mae"]   # Mean Absolute Error to see average % deviation
11 )
```

Figure 17: Configuring the Sequential API and ReLU activation function to define the architecture of the MLP, enabling a regression score. From Cell 5 of *MLP_preferencematch.ipynb*.

The model was compiled using the Adam optimizer and Mean Squared Error as the loss function. Since the architecture consists of only 49 trainable parameters, the training process is computationally efficient. As shown in figure 18, the model is exported directly from the Python memory object using the *tfjs.converters.save_keras_model* function after the training.



```
MLP_preferencematch.ipynb

1 import tensorflowjs as tfjs
2 import os
3
4 # Create the output directory if it doesn't exist
5 output_path = './tmp/mlp'
6 if not os.path.exists(output_path):
7     os.makedirs(output_path)
8
9 # Convert the model directly from the memory object
10 # This avoids the "InputLayer" deserialization bug in the CLI
11 tfjs.converters.save_keras_model(model, output_path)
12
13 print(f"Conversion complete. Files in {output_path}:")
14 !ls -lh {output_path}
```

Figure 18: Using the `tfjs.converters.save_keras_model` function to convert the MLP from the Python memory object, making it ready for import into the Expo application. Results in a `model.json` file and a single binary weight shard.

This approach bypasses known bugs related to deserialization of *InputLayer*, finally producing a `model.json` file alongside a single binary weight shard, ready to be bundled into the Expo application.

11.3.2 Services and model loading

The bridge between the trained model and the mobile application is in the *preferences-Matcher.ts* service. One of the technical challenges of a mobile application like SpisNemt, is loading binary model assets. To solve this, the service uses the `@tensorflow/tfjs-react-native` library along with a validation to ensure the respective `model.json` and `.bin` weights are valid before being initialized.

As shown in figure 19, the model is loaded asynchronously to prevent blocking the main UI in case of delayed or missing imports. This is handled with the `modelPromise` variable, which handles all the heavy lifting in terms of initializing the TensorFlow backend. Loading the layers of the model only happens once during the application lifecycle.



```

78 export async function initPreferencesMatcher() {
79   if (modelPromise) {
80     await withTimeout(
81       modelPromise,
82       INIT_TIMEOUT_MS,
83       "Preference matcher initialization",
84     );
85     return;
86   }
87
88   modelPromise = (async () => {
89     try {
90       validateBundledModelAssets();
91       await tf.ready();
92       return tf.loadLayersModel(
93         tfReactNative.bundleResourceIO(
94           modelJson,
95           modelWeights as unknown as number[],
96         ),
97       );
98     } catch (error) {
99       modelPromise = null;
100       throw new Error(
101         `Failed to initialize preference matcher: ${getErrorMessage(error)}`,
102       );
103     }
104   })();
105
106   try {
107     await withTimeout(
108       modelPromise,
109       INIT_TIMEOUT_MS,
110       "Preference matcher initialization",
111     );
112   } catch (error) {
113     modelPromise = null;
114     throw error;
115   }
116 }

```

Figure 19: Asynchronous model initialization within *preferencesMatcher.ts*, illustrating race-condition mitigation through singleton promise tracking and forced execution time-outs.

11.3.3 Frontend integration and scoring UI

In the home and explore screens, that being *index.tsx* and *explore.tsx*, the *scoreRecipeMatch* function is called after being implemented in *preferenceMatcher.ts* to process recipe data. This can be seen in figure 20.

```

preferencesMatcher.ts

118 export async function scoreRecipeMatch(
119   meal: MealForScoring,
120   prefs: UserPreferences,
121 ): Promise<number> {
122   try {
123     if (!modelPromise) {
124       await initPreferencesMatcher();
125     }
126
127     const currentModelPromise = modelPromise;
128     if (!currentModelPromise) {
129       throw new Error("Preference matcher is not initialized.");
130     }
131
132     const model = await currentModelPromise;
133
134     const areaMatch =
135       prefs.area.length > 0 && prefs.area.includes(meal.strArea ?? "")
136       ? 1.0
137       : 0.0;
138     const catMatch =
139       prefs.category.length > 0 &&
140       prefs.category.includes(meal.strCategory ?? "")
141       ? 1.0
142       : 0.0;
143
144     // 4-float input: [area_match, cat_match, area_norm, cat_norm]
145     const inputData = Float32Array.from([
146       areaMatch,
147       catMatch,
148       areaMatch,
149       catMatch,
150     ]);
151     const inputTensor = tf.tensor2d(inputData, [1, 4]);
152
153     const outputTensor = model.predict(inputTensor) as tf.Tensor;
154     const outputData = await outputTensor.data();
155
156     inputTensor.dispose();
157     outputTensor.dispose();
158
159     return outputData[0] ?? 0;
160   } catch (error) {
161     throw new Error(`Failed to score recipe match: ${getErrorMessage(error)}`);
162   }
163 }

```

Figure 20: Implementation of *scoreRecipeMatch* within *preferencesMatcher.ts*, outlining manual feature engineering, multi-dimensional tensor compilation, and hardware memory disposal routines.

Here, the service receives the recipe along with the user’s preferences, and it then generates a four-input tensor with its match signals. After this, as shown in 21, the tensor is passed to the MLP, which returns a continuous score between 0 and 1. In the UI defined in (*tabs*)/*index.tsx*, as seen in figure 21, the scores are stored in a Map and passed to the RecipeCard component:

```

46 React.useEffect(() => {
47   if (randomMeals.length === 0 || !userPrefs) return;
48   if (userPrefs.area.length === 0 && userPrefs.category.length === 0) return;
49
50   void (async () => {
51     const entries = await Promise.all(
52       randomMeals.map(async (meal) => {
53         try {
54           const score = await scoreRecipeMatch(meal, userPrefs);
55           return [meal.idMeal, score] as [string, number];
56         } catch {
57           return [meal.idMeal, 0] as [string, number];
58         }
59       }),
60     );
61     setMatchScores(new Map(entries));
62   })();
63 }, [randomMeals, userPrefs]);

```

Figure 21: Code snippet from the *(tabs)/index.tsx* component layer, showing asynchronous batch processing of model weights and matching properties mapping.

After setting the `matchScore` property, it is passed to the `RecipeCard` component.

```

111 <Scrollable horizontal>
112   {(hasPrefs ? recommendedMeals : randomMeals).map((meal) => (
113     <Pressable key={meal.idMeal} onPress={() => openMeal(meal)}>
114       <RecipeCard
115         title={meal.strMeal}
116         category={meal.strCategory}
117         imageUrl={meal.strMealThumb}
118         matchScore={matchScores.get(meal.idMeal)} // Passing the score
119       />
120     </Pressable>
121   ))}
122 </Scrollable>

```

Figure 22: Code snippet from *(tabs)/index.tsx*, where the inference score is extracted from the reactive `matchScores` map and passed via the `matchScore` property into the `RecipeCard` component.

This card translates the values into visual "Perfect Match," "Good Match," or "Poor Match" badges based on the thresholds defined in the design section 10. These badges can be seen in figures 23 and 24.

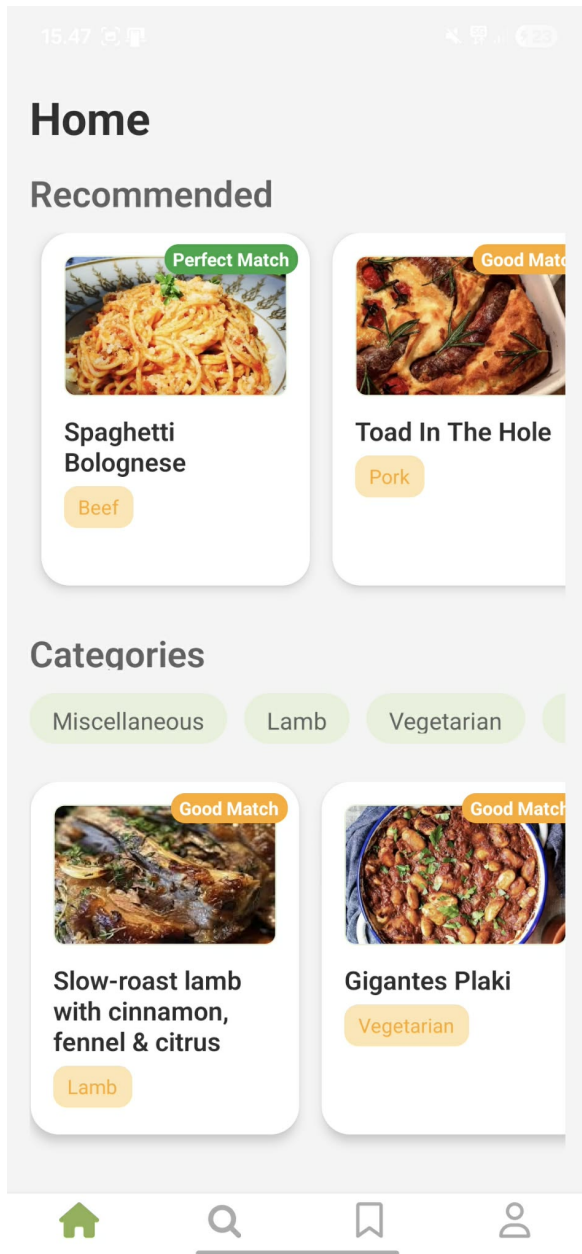


Figure 23: Home screen with "perfect" and "good" matches

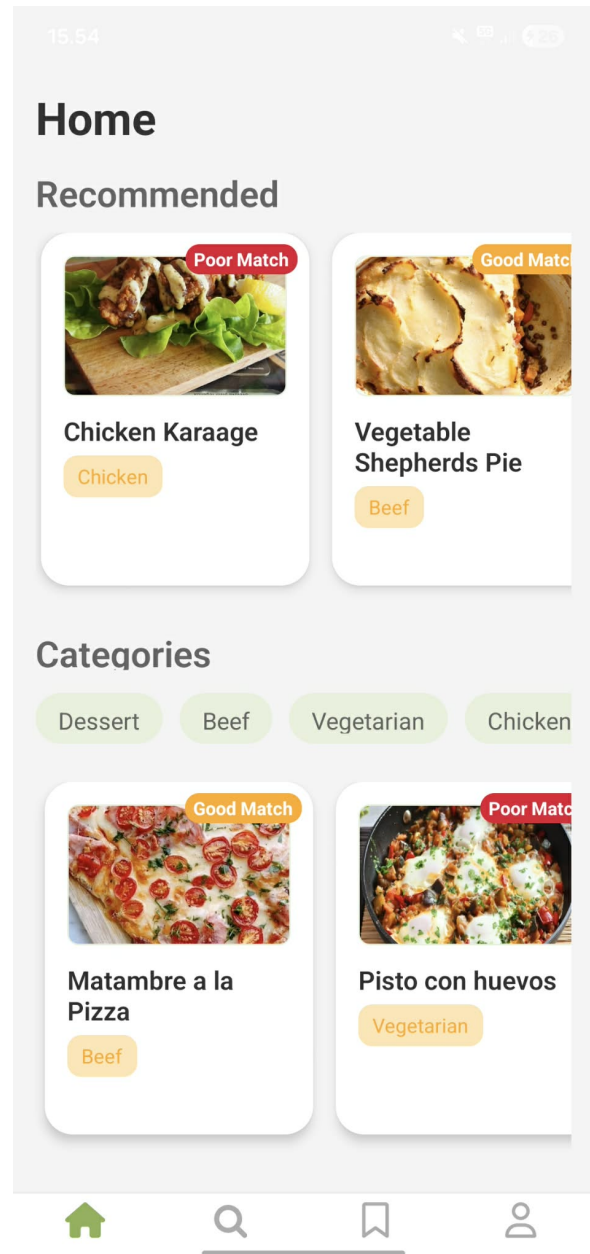


Figure 24: Home screen with "good" and "poor" matches

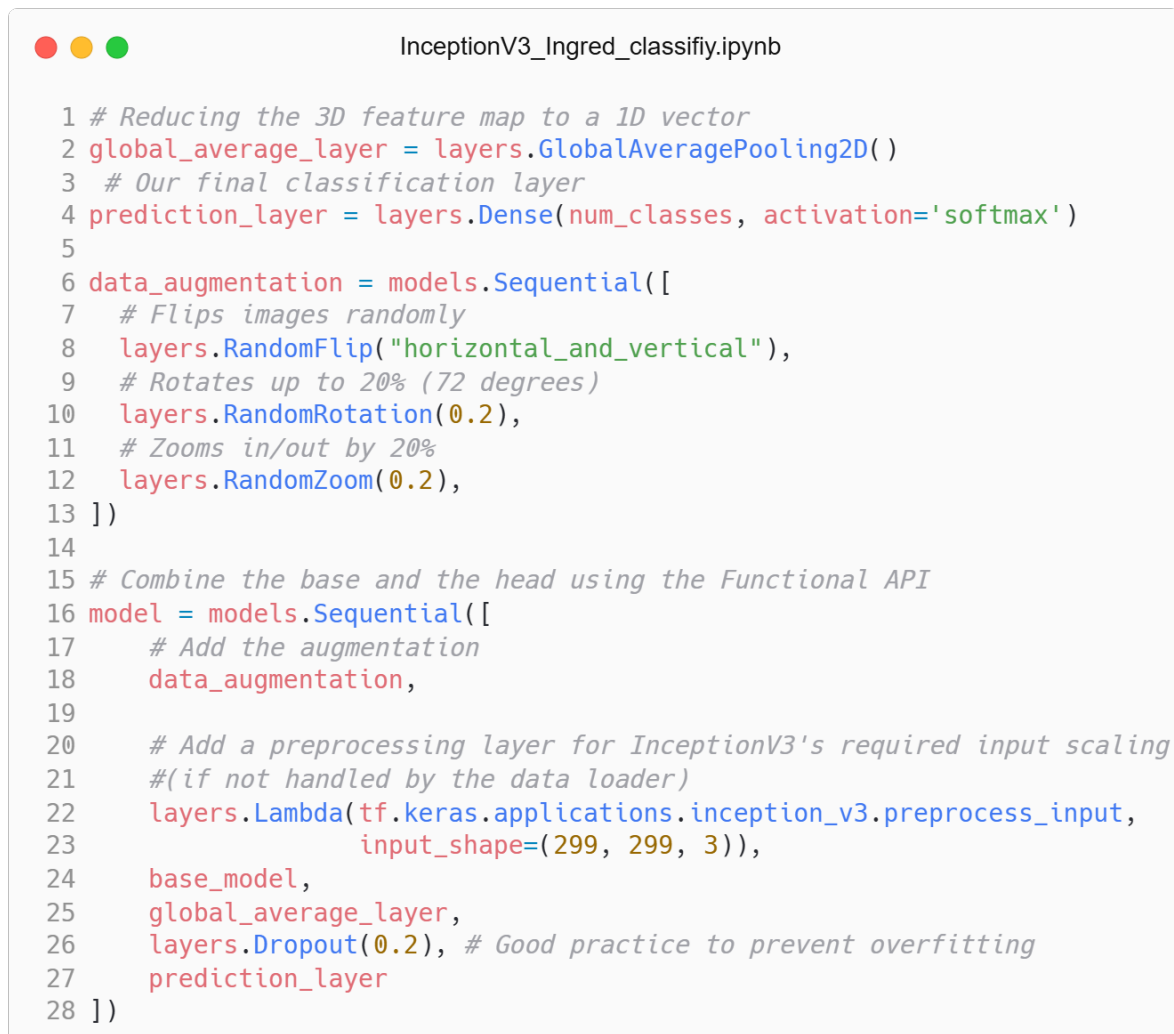
11.3.4 Ingredient classification

Besides recipe matching, the system uses a Convolutional Neural Network to handle ingredient registration through the device's camera. This functionality realizes the Computer Vision component defined in the C3 backend design specified in section 10. The implementation is defined in the *ingredientClassifier.ts* service, which manages the pretrained InceptionV3 model within TensorFlow.js. The initial setup before starting the training, involves setting 3 constants; The IMG_SIZE of 299x299, the BATCH_SIZE of 32, the VAL_SPLIT of 0.33 and the EPOCHS of 100.

Transfer learning and pipeline optimization

The ingredient classifier was trained using a transfer learning approach built upon the pretrained InceptionV3 base model. During the Python implementation, the Keras *image_dataset_from_directory* utility is used to load the 15class vegetable dataset. To prevent bottlenecks during training on the GPU, the data pipeline is optimized using *tf.data.AUTOTUNE*, which caches and prefetches batches of the 299x299 pixel images.

To preserve the featureextraction patterns that InceptionV3 learned from the ImageNet dataset, the base model weights are frozen from *base_model.trainable = False*. A custom classification head is then built using the Functional API, as seen in figure 25.



```
1 # Reducing the 3D feature map to a 1D vector
2 global_average_layer = layers.GlobalAveragePooling2D()
3 # Our final classification layer
4 prediction_layer = layers.Dense(num_classes, activation='softmax')
5
6 data_augmentation = models.Sequential([
7     # Flips images randomly
8     layers.RandomFlip("horizontal_and_vertical"),
9     # Rotates up to 20% (72 degrees)
10    layers.RandomRotation(0.2),
11    # Zooms in/out by 20%
12    layers.RandomZoom(0.2),
13 ])
14
15 # Combine the base and the head using the Functional API
16 model = models.Sequential([
17     # Add the augmentation
18     data_augmentation,
19
20     # Add a preprocessing layer for InceptionV3's required input scaling
21     #(if not handled by the data loader)
22     layers.Lambda(tf.keras.applications.inception_v3.preprocess_input,
23                   input_shape=(299, 299, 3)),
24     base_model,
25     global_average_layer,
26     layers.Dropout(0.2), # Good practice to prevent overfitting
27     prediction_layer
28 ])
```

Figure 25: Code snippet from *InceptionV3_ingred_classify.ipynb*, showing the sequential integration of localized data augmentation layers, tensor preprocessing, frozen InceptionV3 base nodes, and the final classification head layers.

To further improve the robustness of the model against real user photography, a data augmentation layer is injected first, by applying random horizontal and vertical flips, 20 percent rotation, and 20 percent zooming. The frozen base model then processes these augmented images, and the *GlobalAveragePooling2D* layer flattens it. Finally, a Dropout layer set to 0.2 is implemented to prevent overfitting before the tensor reaches the final

Dense classification layer, which uses a softmax activation function to output probability distributions for the 15 target ingredient classes. After training, this architecture is converted to the TensorFlow.js format, generating the 21 binary weight shards for client-side prediction.

11.3.5 Model initialization

A technical hurdle of the machine learning is the hardware overhead of loading large model weights. The InceptionV3 architecture used in SpisNemt is more complex than the MLP, requiring bundling and validation of 21 separate binary weight shards. To maintain a responsive UI and adhere to the requirements in 6.2, the service utilizes a *modelPromise* pattern to ensure the *tf.loadGraphModel* execution remains asynchronous as shown in figure 26.



```
ingredientClassifier.ts

54 const modelJson = require("../ml/CNN/content/tfjs_incep_model/model.json");
55 const modelWeights = [
56   require("../ml/CNN/content/tfjs_incep_model/group1-shard1of21.bin"),
57   require("../ml/CNN/content/tfjs_incep_model/group1-shard2of21.bin"),
58   require("../ml/CNN/content/tfjs_incep_model/group1-shard3of21.bin"),
59   require("../ml/CNN/content/tfjs_incep_model/group1-shard4of21.bin"),
60   require("../ml/CNN/content/tfjs_incep_model/group1-shard5of21.bin"),
61   require("../ml/CNN/content/tfjs_incep_model/group1-shard6of21.bin"),
62   require("../ml/CNN/content/tfjs_incep_model/group1-shard7of21.bin"),
63   require("../ml/CNN/content/tfjs_incep_model/group1-shard8of21.bin"),
64   require("../ml/CNN/content/tfjs_incep_model/group1-shard9of21.bin"),
65   require("../ml/CNN/content/tfjs_incep_model/group1-shard10of21.bin"),
66   require("../ml/CNN/content/tfjs_incep_model/group1-shard11of21.bin"),
67   require("../ml/CNN/content/tfjs_incep_model/group1-shard12of21.bin"),
68   require("../ml/CNN/content/tfjs_incep_model/group1-shard13of21.bin"),
69   require("../ml/CNN/content/tfjs_incep_model/group1-shard14of21.bin"),
70   require("../ml/CNN/content/tfjs_incep_model/group1-shard15of21.bin"),
71   require("../ml/CNN/content/tfjs_incep_model/group1-shard16of21.bin"),
72   require("../ml/CNN/content/tfjs_incep_model/group1-shard17of21.bin"),
73   require("../ml/CNN/content/tfjs_incep_model/group1-shard18of21.bin"),
74   require("../ml/CNN/content/tfjs_incep_model/group1-shard19of21.bin"),
75   require("../ml/CNN/content/tfjs_incep_model/group1-shard20of21.bin"),
76   require("../ml/CNN/content/tfjs_incep_model/group1-shard21of21.bin"),
77 ];
```

Figure 26: Importing the model and the 21 bundled and validated model weight shards

Once the model and its weights are imported, the function *initIngredientClassifier* is implemented.



```

130 export async function initIngredientClassifier() {
131   if (modelPromise) {
132     await withTimeout(
133       modelPromise,
134       INIT_TIMEOUT_MS,
135       "Ingredient classifier initialization",
136     );
137     return;
138   }
139
140   modelPromise = (async () => {
141     try {
142       validateBundledModelAssets();
143       await tf.ready();
144       return tf.loadGraphModel(
145         tfReactNative.bundleResourceIO(
146           modelJson,
147           modelWeights as unknown as number[],
148         ),
149       );
150     } catch (error) {
151       modelPromise = null;
152       throw new Error(`Failed to initialize ingredient classifier:
153         ${getErrorMessage(error)}`);
154     }
155   })();
156
157   try {
158     await withTimeout(
159       modelPromise,
160       INIT_TIMEOUT_MS,
161       "Ingredient classifier initialization",
162     );
163   } catch (error) {
164     modelPromise = null;
165     throw error;
166   }
167 }

```

Figure 27: Initializing the ingredient classifier, using *tf.loadGraphModel* in *modelPromise* to asynchronously load the model and its weights model weights.

In the loading logic, there is a 45 second timeout implemented specifically for the module to compensate for the warm up of the graph model, which comes from the mobile GPU acceleration.

11.3.6 The data pipeline

Bridging the `CameraView` of `React Native` in the `ExploreScreen` and the mathematical model, is a data transformation pipeline in the function `classifyIngredientFromUri`. As seen in figure 28, when a user captures a photo, the image is provided as a local URI, which is incompatible with `TensorFlow`'s tensor-based math. The implementation must manually bridge this gap by converting the file into a standard `float32` tensor of $299 \times 299 \times 3$ pixels as required by the `InceptionV3` input layer.

```
ingredientClassifier.ts

196 export async function classifyIngredientFromUri(imageUri: string): Promise<IngredientPrediction> {
197   try {
198     if (!modelPromise) {
199       await initIngredientClassifier();
200     }
201
202     const currentModelPromise = modelPromise;
203     if (!currentModelPromise) {
204       throw new Error("Ingredient classifier is not initialized.");
205     }
206
207     const model = await currentModelPromise;
208
209     const imageData = await readImageBytesFromUri(imageUri);
210
211     const imageTensor = tfReactNative.decodeJpeg(imageData, 3);
212     const inputTensor = tf.tidy(() => {
213       // InceptionV3 expects [1, 299, 299, 3].
214       return tf.image.resizeBilinear(imageTensor, [299, 299]).expandDims(0).toFloat();
215     });
216
217     const outputTensor = model.predict(inputTensor) as tf.Tensor;
218     const outputData = await outputTensor.data();
219
220     imageTensor.dispose();
221     inputTensor.dispose();
222     outputTensor.dispose();
223
224     let maxIndex = 0;
225     for (let i = 1; i < outputData.length; i += 1) {
226       if (outputData[i] > outputData[maxIndex]) {
227         maxIndex = i;
228       }
229     }
230
231     const label = MODEL_LABELS[maxIndex] ?? `class_${maxIndex}`;
232     return {
233       label,
234       normalizedTerm: labelToSearchTerm(label),
235       confidence: outputData[maxIndex] ?? 0,
236     };
237   } catch (error) {
238     throw new Error(
239       `Failed to classify ingredient image: ${getErrorMessage(error)}`,
240     );
241   }
242 }
```

Figure 28: Code snippet of the asynchronous `classifyIngredientFromUri` core routine within `ingredientClassifier.ts`, showing automated binary JPEG array transformations, input shape matrices scaling, and localized tensor memory tracking.

This process of transforming involves reading the bytes into an array before decoding them into a three channel tensor for JPEG. Standardization is done via *tf.image.resizeBilinear*, ensuring that the micro-details of the ingredients are preserved, despite the downscaling.

11.3.7 UI Handshake

Once the tensor is fed into the predict function, the implementation must translate the raw numerical output back into a user-readable format. The service identifies the index with the highest probability score and maps it against a pre-defined "MODEL_LABELS" manifest. This returns a normalized search term such as "Basil" or "Onion" directly into the state defined in *Explore.tsx*. By implementing this logic on the client side, the system closes the loop from the kitchen environment to the MealDB database without the latency of a cloud-based API.

11.4 Section summary

To conclude, the implementation phase translates the C4 architectural models into a functional, component-driven React Native application. By utilizing the Firebase JS SDK for secure user management and TensorFlow.js for localized, asynchronous machine learning inference, the system maintains high performance and UI responsiveness.

12 Test

Following the implementation of the SpisNemt system, a testing phase was initiated. The objective of this phase is to systematically verify that the application fulfills the functional requirements and user stories defined in section 6.1.1. Due to the project's time constraints and the complexity of the on-device machine learning modules, the testing strategy focuses strictly on verification of the user flows over automated unit testing. The system's integrity is validated through formal Acceptance Testing.

12.1 Acceptance test

The acceptance test is conducted on the 5th of May 2026 at 10 AM with all group members present, spanning roughly 35 minutes. The test is of the epics and user stories in section 6.1.1. The acceptance test is seen in table 10, where the results are stated in the "Results" column. The results are scored in two different ways; "Pass" if all criteria are met and "Fail" if not all criteria are met.

The acceptance test is performed by the group members with supervisor, Carsten Eie Frigaard, observing. A group member is responsible for reading each test scenario aloud along with the corresponding action and acceptance criteria. This member also checks the firebase database for potential relevant updates, which is an acceptance criteria in three of the nine tests. The other member is responsible for performing the test which is done with a development build on an physical android device. Actions and observations are made for each test, which can be seen in [Appendix Results](#).

Test phase summary

By running these scenarios in a live native environment, the Acceptance Test effectively stress-tests the integration between the React Native frontend, the local TensorFlow models, and the Firebase backend.

13 Results

This section presents the outcomes of the system evaluation, divided into two parts; the Acceptance Test conducted on the physical device and the mathematical performance metrics of the machine learning module, run in an isolated environment.

13.1 Acceptance test

Table 10 provides an overview of the system's compliance with the predefined epics and user stories. Out of nine test scenarios, seven successfully met all acceptance criteria.

The two scenarios that failed were as follows:

- **Test 5 (Epic 2):** The ingredient classifier failed to accurately register ingredients using the device's camera.
- **Test 9 (Epic 3):** The system failed to provide a refresh mechanism for the recipe results.

A comprehensive action log detailing the specific user inputs and the observations for each test is shown in [Appendix Results](#).

No.	Epic	User story	Action	Acceptance criteria	Result
1	Epic 1	A user must be able to create a new account	A user creates a new account through the form in the <i>login</i> screen	The newly created account must be visible in the firebase database and the user is automatically logged in	Pass
2		A user must be able to log out of their account	A user logs out of their account through the <i>account</i> screen	The user is redirected to the <i>login</i> screen	Pass
3		A user must be able to log in to their existing account from the <i>login</i> screen	A user logs in to their existing account	The user is redirected to the <i>home</i> screen	Pass
4		A user must be able to register preferences (cuisine & category) through the <i>account</i> screen after user registration	The user edits their preferences (cuisine & category) in the <i>account</i> screen to Cuisine: Italian, French, Thai, Spanish Category: Chicken, Beef, Pasta	The preferences (cuisine & category) are updated in the firebase database so it matches the entered preferences	Pass
5	Epic 2	A user should be able to register ingredients as search terms using the device's camera	The user scans food items using the camera in the <i>explore</i> screen	The scanned ingredients are placed underneath the search bar as search terms	Fail
6		A user must be able to register ingredients as search terms through text in an input field in the <i>explore</i> screen	The user writes an ingredient in the search field and presses <i>space-bar</i> to add it as a search term	The written ingredients are placed underneath the search bar as search terms	Pass
7	Epic 3	A user must be able to save recipes to the <i>saved recipes</i> screen	The user presses a recipe in the home screen. After the recipe opens, the user presses the <i>bookmark</i> icon	The id for the recipe is visible in the firebase database and the recipe is present in the <i>saved recipes</i> screen	Pass
8		When searching for recipes, the results must be filtered based on the user's preferences (cuisine & category)	The user submits the search terms "onion", "garlic", and "potatoes" in <i>explore</i> screen	Recipes with the highest match are shown first which is highlighted with the match batches (Perfect, good, poor) then no batch	Pass
9		When searching for recipes, the user must be presented with 4 results at a time, allowing the user to refresh the results	The user submits the search terms "onion", "garlic", and "potatoes"	The user is presented with four recipes and is able to refresh to be shown four new recipes	Fail

Table 10: Evaluation table showing Acceptance Testing parameters, execution flows, and functional compliance states.

13.2 Ingredient Classifier Performance

To validate the mathematical performance of the transfer learning approach, the Computer Vision module was tested and implemented in an isolated Jupyter environment on a GPU cluster. The model was trained over 100 epochs with a batch size of 128, a validation split of 0.33, and using a dataset from Kaggle of 15,000 images across 15 classes [25].

The results from this validation show almost perfect classification capabilities under the controlled conditions of the Jupyter environment, as illustrated in figure 29.

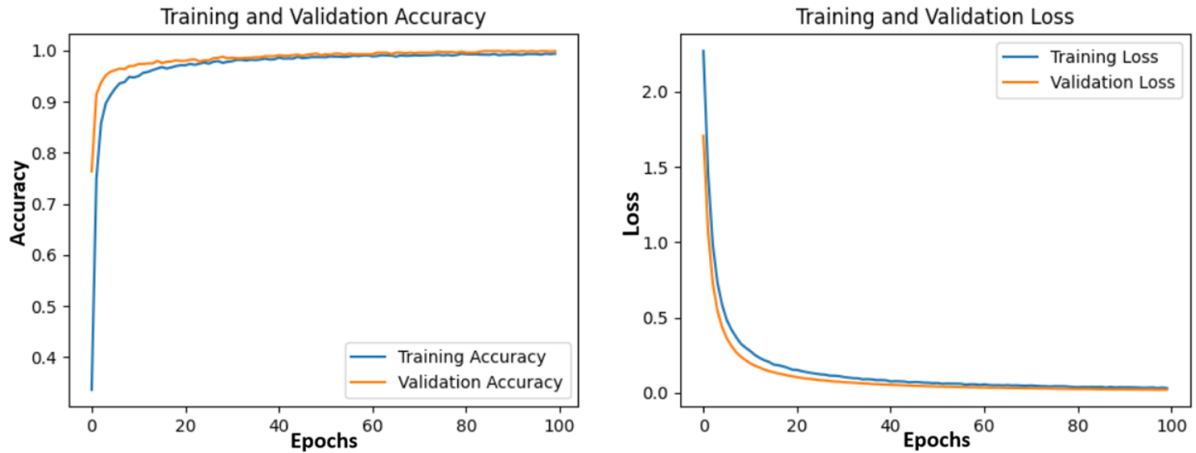


Figure 29: Graphs showing validation accuracy and validation loss after training the computer vision module in the Jupyter environment.

The metrics shown in figure 29 confirm that the neural network is capable of distinguishing between complex food items on the micro-detail and the macro-feature level, when the input data is clean and consistent with the training distribution.

It is worth noting that in the learning curves of figure 29, the validation loss is slightly lower than the training loss. Typically, validation loss is higher than training loss, but this inverted result is normal and caused by the way the model is trained. During training, data augmentation involving flips, rotations and zooming, and a dropout layer that involves disabling 20% of neurons are applied. This makes it harder for the model to classify the training images, which artificially raises the training loss. During validation, these "hurdles" are no longer applied, and the model evaluates clean images with its full capacity. Additionally, the training loss of the model is calculated as a running average during the epoch, and validation loss is calculated at the end of the epoch after the model has updated its weights. Because of this, the model naturally performs slightly better during the validation step.

13.3 Recipe Recommender Performance

Additionally, to validate the performance of the MLP used for recipe recommendation. This MLP module was implemented and tested in an isolated Jupyter environment on a GPU cluster, just like the Computer Vision module. The model was trained over 15 epochs with a batch size of 128, a validation split of 0.2 and with the dataset generated from the code snippet 16 in section 11.3.1.

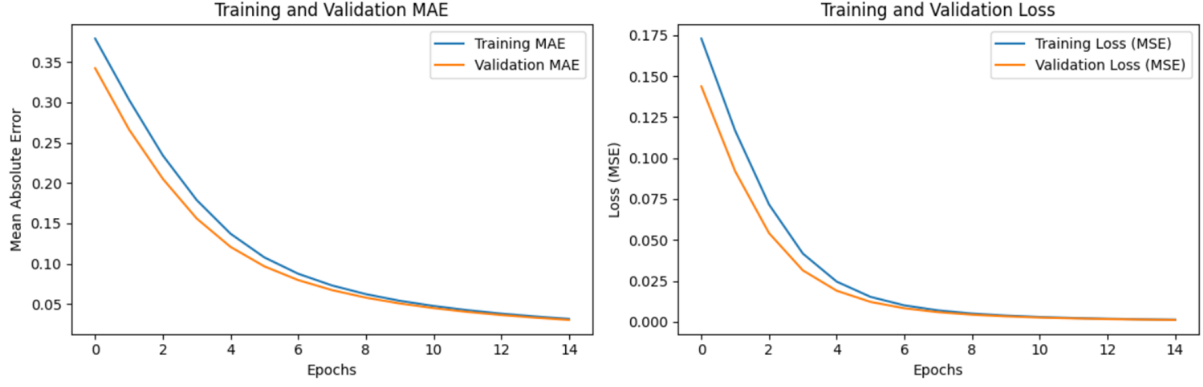


Figure 30: Graphs showing Mean Absolute Error and validation loss after training the MLP recipe recommender module in the Jupyter environment.

The metrics in figure 30 confirm that the MLP efficiently and accurately learns the preference weighting logic. The Mean Absolute Error and the Mean Squared Error both show a smooth intersection over the over its 15 epochs, eventually dropping to nearly zero. The validation curves track the training curves without diverging, and this indicates that the model generalizes exceptionally well to unseen user preference combinations and does not suffer from any overfitting. This demonstrates that the lightweight architecture is capable of predicting accurate match scores.

Similarly to figure 29, there is a small detail in the learning curves of figure 30. Here, the validation loss and the MAE remain slightly below the training metrics. Validation loss is usually higher than training loss, but this inversion is a result of how the training process is timed. For this MLP, the training loss is calculated as a moving average while the weights of the network are continually being updated across the batches. As opposed to this, the validation metrics are calculated at the end of the epoch. This means the validation set is evaluated by a model that has just had a full round of learning, meaning that it is slightly more accurate than the model used to calculate the training loss. Lastly, since the dataset is randomly generated, it is likely that the validation split contains a set of user preferences that are easier for the network to predict.

13.4 Section summary

In summary, the evaluation proves that SpisNemt’s core infrastructure is sound, successfully passing seven out of nine acceptance criteria. However, a gap remains between the model’s high accuracy in isolation and its live performance on a physical device, where camera registration failed. Ultimately, the system foundations are stable, but the live hardware-to-model integration requires further optimization.

14 Discussion

This section provides a critical analysis of the SpisNemt system, looking into the actual outcomes as opposed to the original architectural objectives. While the user journey was successfully implemented, translating the ML models into a live native environment introduced technical bottlenecks. The following section elaborates on the acceptance test failures, as stated in the previous section, reflects on the compromises made regarding testing strategies, evaluates the hardware and software integration gap of the machine learning modules, and critically reviews the system's offline capabilities. Discussions of Planned End-to-End test, Testing strategy reflection, Expo difficulties, Database reflection, Authentication difficulties are found in [Appendix Discussion](#).

14.1 Acceptance test results

From the acceptance test in section 13.1, it can be seen that seven out of nine test scenarios pass, as their acceptance criteria were met. The core user flow, from registration to recipe recommendation, is functional. However, this flow only works correctly when entering ingredients through text, as the acceptance test number 5 from section 6.1.1 failed, where an apple was incorrectly identified as a cabbage. Additionally test number nine, as seen in table 10, failed because no refresh option is available in the interface of the app. Actions and observations of test 5 and 9 can be found in table 11, where the full table for all tests are found in [Appendix Results](#).

The failure of test 5 highlights a discrepancy between the model's performance in the Jupyter environment and its operation within the mobile application. More details on the model's poor performance on the mobile application are analyzed in depth in section 14.3. Test number 5 tested Epic 2, user story 1 from section 6.1.1, which is a *should* requirement. Because of the requirement being a *should* in terms of MoSCoW, this user story was not prioritized as much as all other user stories, as they are *must* requirements. By being able to input ingredients through text in an input field, as seen in test number 6 in table 10, it is still possible to complete the user journey from user registration to being shown recipes based on available ingredients.

No.	Epic	Action/Observation
5	Epic 2	The user presses the explore screen in the navigation bar then presses the camera button, where a camera window appears in the explore screen. A picture of an apple on a computerscreen is taken with the phone camera (https://www.walmart.com/ip/Fresh-Red-Delicious-Apple-Each/45912954). The app processes the image for 30-60 seconds. The results is then given where it reads "Detected cabbage (17.5%)" Cabbage is added as a search term underneath the text input field.
9	Epic 3	In the explore screen the user removes the search terms "apple" and "cabbage" by pressing on them. 3 new search terms are added: "onion", "garlic", and "potatoes". Enter is pressed to start the search. 5 recipes appear with no way to refresh the search results.

Table 11: Snippet of Action Log overviewing the Acceptance test and highlighting execution data for Test Scenarios 5 and 9.

Test number 9 failed the acceptance criteria of showing four recipes with the ability to refresh, because the search results contained more than four recipes with no option to refresh, as stated in table 10. As user story 3 in Epic 3, from section 6.1.1 was deemed to not be as important for the apps overall user flow as the other user stories, this user story was chosen to not be implemented. If implemented, a problem would have arisen with the refresh functionality, because TheMealDB only has around 600 recipes. When fetching with multiple ingredients, the returned recipes are few and sometimes less than a total of four. This causes the refresh functionality to be obsolete as you will not be able to fetch new recipes. TheMealDB was the best option for a public API where it was possible to fetch recipes based on multiple ingredients.

A significant technical achievement was the recipe recommender which uses the users preferences and available ingredients to return corresponding recipes. This part of the app works as intended by filtering the returned recipes based on how well they match the users preferences and available ingredients. This step of the user journey ties together other parts of the epic and user stories as well as the acceptance test in the sense that other user stories, such as test 4, and test 5 or test 6, has to work in order for test 8 in table 10 to work.

If test 5 were to pass the acceptance criteria the app would feel more complete, as the flow of scanning ingredients is more in line with the projects problem statement in section 5.1. The user story from test 6 was created to ensure the the user journey from user registration to receiving a recipe was possible in the event that scanning ingredients did not work as intended.

14.2 Model architecture

An important architectural decision made, is the choice of implementing the ML models as internal mechanisms using TensorFlow.js rather than external services called via RESTful APIs. This choice is backed by the requirement for responsive predictions given in 6.2, from camera data to classification to recipe matching, without any latency costs coming from server-side operations. By running the models on the native hardware, in theory, the system gains benefit in terms of scalability, as the burden is spread across all potential users' devices rather than on a single backend.

14.3 The gap between train and test

While there are benefits of running the image classification on the users's device, the InceptionV3 model showed a gap in generalization when run on mobile devices as opposed to the Jupyter notebook where it was trained and tested. In the Jupyter notebook, as documented in the quantitative results in section 13, the classifier showed high precision, reaching a final validation accuracy of 99.89%. However, in a mobile application context, the predictions often defaulted to specific classes, such as "Potato", "Bean" and "Broccoli", most often with accuracies of 16%.

The probable causes of this are varying. One being quantization, another being linked to the pre-processing of the model, and the last probable cause is environmental noise. Converting the model to a web-friendly format through TensorFlow.js involved quantization to make the model more lightweight in terms of resource consumption. This led to a reduction in precision. Additionally, a mismatch in the image pre-processing libraries

in Python and the scaling mechanisms of the React Native camera module introduced artifacts that the model was not trained to handle. Lastly, the dataset that the model was trained on [25] contains images from many different environments, but mainly the pictures are taken in grocery stores. Therefore, kitchen lighting and background clutter differs from the balanced dataset used during transfer learning, which potentially leads to mismatches or model "confusion".

The decision to use TensorFlow.js as the runtime for ML was backed by the requirements for prediction time and scalability. By choosing a single framework that supports CNN and MLP architectures natively, the system remained lightweight, satisfying the performance requirements and the supportability requirements of section 6.2. While the integration gap remains an unsolved challenge for live inference, the modular success confirmed that the underlying architectural choices were mathematically sound.

14.4 Reflection on offline capabilities

While offline reliability is defined as a requirement in section 6.2.2 it has not been fully implemented as development time has come to an end. The non-functional requirements sections states two offline reliability requirements, one being the ability to scan ingredients when offline and the other being the ability to view saved recipes when offline. Scanning ingredients while offline is possible as the machine learning models are local models which run on the device and does not need to contact a server. However while scanning ingredients when offline possible, the following step of searching for a recipe is not possible as it requires a connection to TheMealDB. The InceptionV3 model does not work optimally as discussed in section 14.3.

Viewing saved recipes when offline does not work entirely because in order to fetch the saved recipes, the database must first return the id's of the recipes and then a GET request must be made to the TheMealDB. If internet connection is lost after GET'ing the recipes, the recipes will still be available long as the application is not closed or reloaded.

Another issue is that the application uses accounts, meaning users must log in to their account in order to access the app. This cannot be done when offline as the login sequence requires a connection to Cloud Firestore.

Looking back the requirement of being able to view recipes when offline, should have been worded differently along the lines of "The user must be able to view their saved recipes when internet connection is lost while logged in". Additionally a requirement of updating preferences when logged in and internet connection is lost could have been made.

The offline requirements from section 6.2.2 could have been omitted as the need to login requiring internet, causing offline requirements inside the app to not be necessary.

14.5 Omitted requirements

As stated in the requirements section, some requirements originally made for the system were omitted. These requirements can be seen in [Appendix Requirements](#). The requirements are related to the scoring system of Lighthouse 10, which is an auditing tool integrated into browser's DevTools. These requirements are to score the usability and performance of the project, through different metrics provided by Lighthouse. However with a tech stack focusing on mobile development, Lighthouse scoring is irrelevant as

mobile applications does not use a browser. Other alternatives for performance testing are researched, where the best choice is the closed beta version of EAS Observe. Using a closed beta is not an option, as there is no guarantee for access. The requirements related to Lighthouse were omitted from the project because of this no promising alternative is found.

14.6 ErrorBoundary

The requirements of the UI must practice error prevention to minimize the relevance of error handling and the system must provide clear error messages, from section 6.2.1, are not fully implemented, as the implementation of 11.2 is only a basic implementation of error containment. The error messages provided from *ErrorBoundary* are limited and not always relevant for the user, thus not entirely living up to these non-functional requirements.

Additionally, the requirement of the system must be fault tolerant on a UI level, having the ability to continue when one or more components fail, is not completely implemented and does not live up to the requirements. While the global *ErrorBoundary* does improve resilience against specific failures, such as errors thrown during React render, it does not cover all errors which can occur.

These non-function requirements are not fully implemented, as development time was limited, thus prioritizing other requirements such as the user stories from section 6.1.

14.7 Section summary

In summary, the development of the SpisNemt system required navigating trade offs between feature implementation and mobile constraints. The successful deployment of the MLP recommender shows viability of on-device prediction for responsive user experiences. Although, the struggles with the Inception V3 model and the limitations of offline data synchronization highlight the challenges of edge computing and external API dependencies. Additionally error prevention and error messages was not prioritized, due to development time restrictions. Furthermore, the initial authentication system as well as the second choice ended up not working, leading to a custom solution. Despite these hurdles, the system successfully proves the core concept of a decoupled, machine learning driven recipe application, setting a foundation for future iteration.

15 Conclusion

This thesis has documented the process, engineering, design, and implementation of a mobile client-side machine learning application aimed at reducing household food waste.

The process and methodology used throughout the project aligned well for both group members, as official Scrum framework guidelines would not have been possible to comply with due to team size constraints. The custom scrum usage, with its iterative process, successfully guided the planning and development of the project. In addition, the GANTT model and ASE model supported the process by giving a timeline overview and dividing the entire project into distinct phases. These software methods and processes kept the project going in the correct direction, while establishing an environment for collaborative discussions when technical problems arose.

The requirements made for this project used functional as well as non-functional requirements, where the functional requirements were graded with respect to MoSCoW and user stories and the non-functional were made with the FURPS framework. Creating the functional requirements as user stories proved to be a significant asset for the acceptance test, as the test scenarios were then pre-established. The non-functional requirements caused issues, as after creating the requirements and having finished the technological analysis, it was evident that numerous requirements could not be used. In the end, the usability requirements are not met as well as the offline reliability requirements.

To bridge the gap between these requirements and development, the ASE models elaboration phase used low fidelity sketches to establish a layout and modular front-end components before implementing any code. Architecturally, the system was detailed from level C1 to C3 of the C4 model framework. This architectural design established a separation of concern, where the UI container handles user navigation and the six different screens, while the Backend container handles authentication along with the machine learning models for client-side inference.

An evaluation of the project was done through an acceptance test, where the system passed seven of the nine pre-defined tests. Those not passing being the InceptionV3 model, not performing as it did in the training environment, and the lack of refresh ability when presenting the user with recipes. However, the entire user journey from initial user registration to being presented ranked recipes, based on preferences and available text-inputted ingredients remain functional. This concludes the acceptance test successful on seven out of nine criteria.

The most prominent problems encountered throughout the project was the authentication system and the integration of the InceptionV3 model for client-side use. The authentication system changed twice in development and eventually a correct and working implementation was made. The integration of the InceptionV3 model was possible but the model did not perform the same as it did in the training environment.

Overall, the problem statement of examining how to design and implement an app, which allows users to scan ingredients and receive a recipe is successful. A fully working app is definitely possible as only one major problem kept SpisNemt from meeting this goal. By using SpisNemt, a user is recommended recipes based on their available ingredients from their kitchen as well as their preferences, hereby having the ethical choice to use their remaining ingredients and not impact household food waste, supporting sustainability.

16 Future Work

Future work is found in [Appendix Future work](#)

References

- [1] United Nations Climate Change. *Food loss and waste account*. 2023. URL: <https://unfccc.int/news/food-loss-and-waste-account-for-8-10-of-annual-global-greenhouse-gas-emissions-cost-usd-1-trillion> (visited on 01/25/2026).
- [2] UNDP. *United nations Goal 12.3*. 2025. URL: <https://sdgs.un.org/goals/goal12#targets-and-indicators> (visited on 02/03/2026).
- [3] Google. *Lighthouse Accessibility Scoring*. 2025. URL: <https://developer.chrome.com/docs/lighthouse/accessibility/scoring> (visited on 01/29/2026).
- [4] Google. *Lighthouse Best Practices Scoring*. 2019. URL: <https://developer.chrome.com/docs/lighthouse/best-practices/doctype> (visited on 01/29/2026).
- [5] Google. *Lighthouse Performance Scoring*. 2019. URL: <https://developer.chrome.com/docs/lighthouse/performance/performance-scoring> (visited on 01/29/2026).
- [6] scrum.org. *ScrumRetrospective*. 2026. URL: <https://www.scrum.org/resources/what-is-a-sprint-retrospective> (visited on 04/25/2026).
- [7] scrum.org. *Scrum*. 2026. URL: <https://www.scrum.org/> (visited on 04/25/2026).
- [8] Figma. *Figma: The Collaborative Interface Tool*. 2026. URL: <https://www.figma.com/> (visited on 05/09/2026).
- [9] Dribbble. *Dribbble - Discover the World's top Designers & Creative Professionals*. 2026. URL: <https://dribbble.com/> (visited on 04/02/2026).
- [10] Miro. *AI-innovationsarbejdsområde — Miro*. 2026. URL: <https://miro.com/> (visited on 05/04/2026).
- [11] Vercel, inc. *Next.js*. 2026. URL: <https://nextjs.org/> (visited on 01/30/2026).
- [12] 650 Industries, Inc. *Expo*. 2026. URL: <https://expo.dev/> (visited on 01/30/2026).
- [13] Meta Platforms, Inc. *ErrorBoundary*. 2026. URL: <https://react.dev/reference/eslint-plugin-react-hooks/lints/error-boundaries> (visited on 02/17/2026).
- [14] 650 Industries, Inc. *Expo AuthSession*. 2026. URL: <https://docs.expo.dev/versions/latest/sdk/auth-session/#googleauthrequestconfig> (visited on 03/21/2026).
- [15] 650 Industries, Inc. *Using Google Authentication*. 2026. URL: <https://docs.expo.dev/guides/google-authentication/> (visited on 03/21/2026).
- [16] 650 Industries, Inc. *Using Firebase for Expo*. 2025. URL: <https://docs.expo.dev/guides/using-firebase/> (visited on 02/04/2026).
- [17] 650 Industries, Inc. *Using Supabase for Expo*. 2025. URL: <https://docs.expo.dev/guides/using-supabase/> (visited on 02/04/2026).
- [18] Google LLC. *Offline Persistence*. 2026. URL: <https://firebase.google.com/docs/firestore/manage-data/enable-offline> (visited on 05/12/2026).
- [19] Invertase. *Offline Capabilities*. 2026. URL: <https://rnfirebase.io/firestore/usage#offline-capabilities> (visited on 02/04/2026).
- [20] Pete Warden. *How many images do you need to train a neural network?* 2017. URL: <https://petewarden.com/2017/12/14/how-many-images-do-you-need-to-train-a-neural-network/> (visited on 02/24/2026).
- [21] Simon Brown. *The C4 model for visualising software architecture*. 2026. URL: <https://c4model.com> (visited on 03/02/2026).
- [22] Simon Brown. *Code diagram*. 2026. URL: <https://c4model.com/diagrams/code> (visited on 03/02/2026).
- [23] Simon Brown. *C4 Model notation*. 2026. URL: <https://c4model.com/diagrams/notation> (visited on 03/02/2026).

- [24] Google LLC. *Add data to Cloud Firestore*. 2026. URL: <https://firebase.google.com/docs/firestore/manage-data/add-data> (visited on 03/05/2026).
- [25] M Israk Ahmed. *Kaggle dataset*. 2022. URL: <https://www.kaggle.com/datasets/misrakahmed/vegetable-image-dataset> (visited on 02/11/2026).